

Onyx — UIKit reference

Onyx · multi-process OS on Raspberry Pi 4

The reference of **UIKit** (`SD:/lib/uikit.so`, `user/Kits/uikit`): what it is for, how a program uses it, and every operation it exposes. The operations' part is made from the kit's headers by `tools/docgen/kitdocs.py` — the headers are the source. Overview of all the kits: [The Kits](#).

Contents

1. What it is
2. Using it
3. Index
4. `uikit/uikit.h`
5. `uikit/widget.h`
6. `uikit/root.h`
7. `uikit/win.h`
8. `uikit/canvas.h`
9. `uikit/theme.h`
10. `uikit/text.h`
11. `uikit/label.h`
12. `uikit/button.h`
13. `uikit/dialog.h`
14. `uikit/bmp.h`
15. `uikit/calendar.h`
16. `uikit/checkbox.h`
17. `uikit/codeedit.h`
18. `uikit/colorpick.h`
19. `uikit/combobox.h`
20. `uikit/datagrid.h`
21. `uikit/dropdown.h`
22. `uikit/flat.h`
23. `uikit/font.h`
24. `uikit/global.h`
25. `uikit/groupbox.h`
26. `uikit/icon.h`
27. `uikit/imagebox.h`
28. `uikit/knob.h`
29. `uikit/lang.h`
30. `uikit/layout.h`
31. `uikit/lcd.h`
32. `uikit/listbox.h`
33. `uikit/menu.h`
34. `uikit/numeric.h`
35. `uikit/paint.h`
36. `uikit/panel.h`
37. `uikit/progress.h`

38. `uikit/radio.h`
 39. `uikit/richtextbox.h`
 40. `uikit/scrollbar.h`
 41. `uikit/segmented.h`
 42. `uikit/skin.h`
 43. `uikit/slider.h`
 44. `uikit/splitter.h`
 45. `uikit/sysclip.h`
 46. `uikit/tabhost.h`
 47. `uikit/tabstrip.h`
 48. `uikit/textarea.h`
 49. `uikit/textbox.h`
 50. `uikit/toggle.h`
 51. `uikit/toolbar.h`
 52. `uikit/treeview.h`
 53. `uikit/vpaint.h`
 54. `uikit/vumeter.h`
-

What it is

UIKit is the interface: the windows and their frames, the widgets, the dialogs, the theme, the icons — and the window API itself, `uk_win_*` (`uikit/win.h`, plain C functions: what speaks to the graphics server; a C program links `lib/uikit.imp.c.a`). The rest is in the namespace `uikit`. A window is a `Root`; widgets are added to it; `run ()` is the event loop.

Include	<code>#include "uikit/uikit.h"</code>
Link	<code>lib/uikit.imp.a</code>
Library	<code>SD:/lib/uikit.so</code> — 848 entries in its table (<code>user/Kits/uikit/uikit.abi</code> , append-only)
Sources	<code>user/Kits/uikit/</code>

Using it

A window is a `Root`; widgets are added to it; `run ()` is the event loop. Everything is in the namespace `uikit`.

A window, a label, a button:

```
#include "appkit/appkit.h"
#include "uikit/uikit.h"
using namespace uikit;

static Label *g_label;
static int g_count;

static void onClick (Widget &)
{
```

```

    char text[32]; int n = 0;
    ax_strcat (text, sizeof text, &n, "clicked ");
    char num[12]; ax_itoa (++g_count, num);
    ax_strcat (text, sizeof text, &n, num);
    g_label->setText (text); // the widget redraws itself
}

int main (void)
{
    Root root (320, 120, "Hello"); // a decorated window
    g_label = new Label (12, 12, 296, 20, "Press the button");
    root.addChild (g_label);
    root.addChild (new Button (12, 44, 100, 28, "Click me", onClick));
    root.run (); // until the window is closed
    return 0;
}

```

Dialogs are modal calls:

```

if (uk_messagebox ("Delete", "Delete this file?", MB_YESNO))
{
    char path[256];
    if (uk_file_open (path, sizeof path, "SD:/docs")) // the file chooser
        g_label->setText (path);
}

```

```

unsigned colour = 0x2060C0;
if (uk_color_dialog (&colour, "Pick a colour")) root.setBg (colour);

```

Tabs (TabStrip, uikit/tabstrip.h): a row of closable tabs over the program's own view — the strip shows them, the program shows the chosen one's content. Each tab carries a pointer for the program; a close is only **asked** (the cross, a middle click): the program decides, then removes it.

```

static TabStrip *g_tabs;
static void onTab (Widget &) { show (g_tabs->data (g_tabs->selected)); }
static void onClose (Widget &) { int i = g_tabs->closing; Doc *d = (Doc *) g_tabs->data (i);
                                                                    if (doc_close (d)) g_tabs->remove (i); } // (a
question first, if need be)
static void onNew (Widget &) { g_tabs->select (g_tabs->add ("Untitled", new_doc
()), true); }
...
g_tabs = new TabStrip (0, 0, root.width, 30, onTab);
g_tabs->onClose = onClose; g_tabs->onNew = onNew;
g_tabs->anchor = ANCHOR_LEFT | ANCHOR_TOP | ANCHOR_RIGHT;
root.addChild (g_tabs);

```

setTitle (i, s), setMark (i, on) (a dot: something runs, something new), selectNext (±1) (the program's Ctrl+Tab), activeFace (the chosen tab opens onto the content's colour). The Terminal's tabs.

The theme: draw with the palette (C_BG, C_TEXT, C_ACCENT, C_FACE...), never with fixed colours, so that the program follows the user's theme.

Text in an owner-drawn widget (uikit/text.h): uk_text_fit cuts a line with "...", uk_text_wrap word-wraps a paragraph into lines of a width (Stickies' cards), uk_text_over writes straight on a see-through window — the wallpaper — blended by coverage, with a soft shadow or engraved (the agenda's and Stickies' headers). The tool bar's icons (uikit/toolbar.h, WKT_*) include WKT_TRASH and WKT_PIN:

```
int st[6], ln[6]; bool more;
int k = uk_text_wrap (text, len, 196, 6, st, ln, &more); // up to 6 lines of 196 px
for (int i = 0; i < k; i++) { char l[256]; int n = ln[i] < 255 ? ln[i] : 255;
    memcpy (l, text + st[i], n); l[n] = 0; uk_text (canvas, 22, 33 + 17 * i, l, ink);
}
uk_text_over (canvas, 38, 9, "Pinned notes", 0x00FAFCFF, 2, 1); // bold, a soft
shadow, on the wallpaper
```

Without C++ (uikit/flat.h, in uikit/uikit.h): the same window and widgets as handles, behind plain C functions — what a BASIC program calls (`#import UIKit`), what QBStudio's generated code is made of, and what a C program can use:

```
void *win = uk_window ("Hello", 320, 120, 0);
uk_label (win, 12, 12, 200, 20, "Your name:");
void *name = uk_textbox (win, 12, 36, 200, 24, "", 0);
uk_button (win, 220, 36, 80, 24, "OK", on_ok); /* void on_ok (void *button)
*/
uk_window_run (win); /* until the window is
closed */
```

```
#import UIKit
win = UIKit.window("Hello", 320, 120, 0)
ok = UIKit.button(win, 220, 36, 80, 24, "OK", ADDRESSOF(Clicked))
DO WHILE UIKit.window_wait(win): LOOP
```

`uk_panel (window, x, y, w, h)` is an area with widgets of its own — every maker takes the window *or a panel* as what holds the widget —: a page shown in place of another (`uk_show`), moved elsewhere (`uk_set_parent`). QBStudio's user controls are panels.

A widget added to UIKit that BASIC should reach gets its functions there (a maker, and `uk_set_text / uk_get_value ...` taught its kind).

An icon — any picture ImageKit reads (BMP today, PNG tomorrow), given as icons are drawn (magenta = see-through):

```
#include "uikit/bmp.h"

int w, h;
unsigned *px = ui::icon_load ("SD:/apps/tinypad.app/icon.bmp", &w, &h);
if (px) { /* draw it ... */ delete [] px; }
```

A code editor (uikit/codeedit.h, 2026-10-06 — QBStudio's, made a widget of the kit for every program that edits code: QBStudio, Turtle Quest): BASIC in a monospaced face with its colours, the line numbers, the indentation's guides, a line lit (`hiLine`: an error's, a debugger's current line), the lines with a problem marked, undo / redo, the clipboard, the completion, UTF-8 comments:

```
CodeEdit *ed = new CodeEdit (10, 10, 400, 300);
ed->mono = myMonoFace; // (0: the UI's face)
ed->isKeyword = is_basic_word; // bool (const char *w, int n): the words
coloured and put in capitals
ed->setText ("FOR i = 1 TO 3\n PRINT i\nNEXT\n");
ed->onChange = changed; // Action: the text changed
root.addChild (ed);
ed->hiLine = 2; ed->showLine (1); // line 2 lit and shown (hiLine 1-based,
showLine 0-based)
ed->clearMarks (); ed->addMark (3); // line 3 has a problem
```

The completion: set `complete (void (CodeEdit &, const char *object))`, which calls `ed.addCompletion (name, kind, detail)` for each item offered after `object`. (or Ctrl+Space: `object` empty).

A timer or polling: derive from `Root` and override `onTick ()` (called about 60 times a second).

Several windows (2026-10-07, kapi v94): one more window of the program is a `Root` made with `NewWindow`; the first window's `run ()` serves them all (their events, `onTick`, drawing), a click in one makes it `Root::current ()`, its close box calls `onClose ()` (by default `closeWindow ()`); closing the first window ends the program). Telegram has one a conversation (docs/03 §6 *Several windows*).

```
class ChatWin : public Root
{
public:
    ChatWin (const char *who) : Root (NewWindow (), -1, -1, 500, 400, who) { if (!
win0pened ()) return; /* its widgets */ }
    void onClose () override { closeWindow (); gone = true; } // (deleted later, by
the main window's onTick)
    bool gone = false;
};
ChatWin *w = new ChatWin ("Alice");
if (!w->win0pened ()) { delete w; /* one window only: show it beside */ }
```

An icon in the menu bar's status area (kapi v95): `uk_tray ("SD:/apps/myapp.app/icon.bmp", "My App - 3 new");` a double click on it shows the program's first window again (even minimised) and calls the first `Root`'s `onTray (KAPI_TRAY_OPEN)`, a right click `onTray (KAPI_TRAY_MENU)`; `uk_tray_clear ()` takes it away.

The window API (`uikit/win.h`, 2026-10-08; in `uikit/uikit.h`, or included alone by a C program, which links `lib/uikit.imp_c.a`): the program's windows and what it asks of the graphics server, as plain C functions — `uk_win_create`, `uk_win_present`, `uk_win_on_key`, `uk_win_menu_set`, `uk_win_list`, `uk_win_tray_set`, `uk_win_fullscreen_begin...` (they were AppKit's `kapi_create_window`, `kapi_present...`: docs/03 §5.10.1 has the table). A `Root` calls them for you; a game that draws its own canvas, a full-screen program or a shell program (the dock, the menu bar) calls them itself. Each graphics server has its own `UIKit` (the desktop's speaks to `Elegant`), so the same binary runs on every one; `uk_win_server` says which.

```
#include "appkit/appkit.h"
#include "uikit/win.h"
```

```
static void on_key (unsigned long sender, int ev, gui_value v) { (void) sender; if
(ev == GUI_EVENT_KEY && v == 27) kapi_exit (0); }
```

```
int main (void)
{
    unsigned *fb = uk_win_create (320, 200, "Plasma"); // the canvas: 0x00RRGGBB,
320 a row */
    if (fb == 0) return 1;
    uk_win_on_key (on_key);
    for (int t = 0; !kapi_should_exit (); t++)
    {
        for (int i = 0; i < 320 * 200; i++) fb[i] = (unsigned) (i + t) * 2654435761u
>> 8;
        uk_win_present (); // the server shows it */
        kapi_pump_wait (16); // the events (AppKit's
pump) */
    }
}
```

```

    return 0;
}

```

Index

Everything the headers declare, in their order — the details are in each header's part below.

Name	What it does	Header
<code>uk_len</code>	<code>uk_fh</code>	<code>widget.h</code>
<code>uk_fw</code>	<code>uk_fh</code>	<code>widget.h</code>
<code>uk_fh</code>	<code>uk_fh</code>	<code>widget.h</code>
<code>uk_thumb</code>	The thumb of a trackH px track for <code>total</code> units of content, <code>view</code> of them visible, scrolled to pos, with show false when all fits (or the track is 6 px or les	<code>widget.h</code>
<code>uk_thumb_pos</code>	A cursor offset <code>cy</code> within the track → scroll pos in [0, total-view] (thumb centred).	<code>widget.h</code>
<code>uk_draw_vscroll</code>	Paint the bar (a groove in a shade of <code>bg</code> , the thumb a raised pill) at the right-edge gutter (x,y, w x trackH)	<code>widget.h</code>
<code>UkBarDrag</code>	A list's own scroll bar under the mouse (a widget that draws <code>uk_draw_vscroll</code>)	<code>widget.h</code>
<code>uk_cursor</code>	The pointer's shape (kapi v81, KAPI_CURSOR_*).	<code>widget.h</code>
<code>Action</code>	fired on click/toggle/change; gets the widget	<code>widget.h</code>
<code>Widget</code>	(a type)	<code>widget.h</code>
<code>uk_applet</code>	running as an applet? (its arguments said so)	<code>root.h</code>
<code>uk_pump</code>	the events: the window's (pump_events), an applet's host's	<code>root.h</code>
<code>uk_present</code>	what was drawn shown: <code>uk_win_present</code> , or told to the host	<code>root.h</code>
<code>uk_quit</code>	time to end: the close box, or the host's	<code>root.h</code>

Name	What it does	Header
	AP_CLOSE / its end	
uk_applet_send	a message to the host	root.h
uk_applet_on_message	The host's messages other than AP_PTR / AP_KEY / AP_CLOSE (an applet's own protocol)	root.h
uk_tray	(v95) The program's icon in the menu bar's status area	root.h
uk_tray_clear	(v95) The program's icon in the menu bar's status area	root.h
Root	(a type)	root.h
uk_win_create	The program's first window, a client area of w x h pixels titled title, placed by the server → its canvas (0x00RRGGBB pixels, w a row), 0 on failure (no server)	win.h
uk_win_create_ex	The program's first window at x, y (its frame's top left)	win.h
uk_win_new	Another window of the program (after its first)	win.h
uk_win_select	The window the calls act on (0	win.h
uk_win_destroy	window win (> 0) closed and forgotten (the first: the program's end)	win.h
uk_win_move	its frame moved to x, y (screen coordinates)	win.h
uk_win_resize	Its client area set to w x h, within the canvas it was created with (which stays) → the canvas, 0 no window.	win.h
uk_win_resize2	As uk_win_resize, but the canvas and the frame's copies grow past their first size when needed (new memory at the same addresses)	win.h

Name	What it does	Header
<code>uk_win_resizable</code>	on != 0	<code>win.h</code>
<code>uk_win_alpha</code>	its opacity, 0 .. 255 (fades)	<code>win.h</code>
<code>uk_win_geometry</code>	Its frame's place and size, its client area's size, the work area (the screen less the menu bar and the dock) and its KAPI_WIN_* state into *out → 0, -1 no win	<code>win.h</code>
<code>uk_win_chrome</code>	Its surfaces for the frame UIKit draws (uikit/skin.cpp)	<code>win.h</code>
<code>uk_win_present</code>	its pixels changed: the server shows them	<code>win.h</code>
<code>uk_win_draw_text</code>	One line of text in the kernel's bitmap font at x, y of its canvas, colour c (0x00RRGGBB), the background kept.	<code>win.h</code>
<code>uk_win_on_key</code>	GUI_EVENT_KEY to fn (value: the character or KEY_*)	<code>win.h</code>
<code>uk_win_on_click</code>	GUI_EVENT_CANVAS_CLICK / _MOTION to fn (the old click handler)	<code>win.h</code>
<code>uk_win_on_pointer</code>	the whole pointer stream to fn (GUI_EVENT_PTR_*, drops, WINCTL, WINRESIZE)	<code>win.h</code>
<code>uk_win_cursor</code>	The pointer's shape over its client area (KAPI_CURSOR_*), kept until changed → the shape it had, -1.	<code>win.h</code>
<code>uk_win_cursor_pos</code>	the pointer, relative to its client area	<code>win.h</code>
<code>uk_win_cursor_shown</code>	1 the pointer is drawn, 0 hidden (keyboard only), -1 not known	<code>win.h</code>
<code>uk_win_menu_set</code>	This program's menus (the first window's) and the handler that gets GUI_EVENT_MENU (value	<code>win.h</code>
<code>uk_win_menu_get</code>	For the menu bar	<code>win.h</code>

Name	What it does	Header
<code>uk_win_menu_command</code>	item id sent to the active window (MENU_QUIT: asked to close) → 1 delivered	<code>win.h</code>
<code>uk_win_list</code>	the windows, bottom to top (at most max) → how many	<code>win.h</code>
<code>uk_win_raise</code>	window id to the front (it gets the keys) → 0, -1	<code>win.h</code>
<code>uk_win_close</code>	window id asked to close (as its close box) → 0, -1	<code>win.h</code>
<code>uk_win_minimise</code>	window id (0: the caller's) minimised until raised → 0, -1	<code>win.h</code>
<code>uk_win_place</code>	window id's frame moved to x, y → 0, -1	<code>win.h</code>
<code>uk_win_to_desk</code>	Window id (0	<code>win.h</code>
<code>uk_win_desk</code>	The workspaces	<code>win.h</code>
<code>uk_win_read</code>	The pixels of a rectangle of window id's client area (part 0) or of its frame (1 active, 2 inactive) into dst (stride pixels a row), clipped to it → 0, -1.	<code>win.h</code>
<code>uk_win_apps</code>	The names of the open apps (a window on the current workspace, not a WIN_FLAG_SYSTEM one), one a line → how many.	<code>win.h</code>
<code>uk_win_app_raise</code>	app name's window to the front → 1, 0 not running / no window	<code>win.h</code>
<code>uk_win_app_toggle</code>	App name toggled → 0 it was running and is asked to close, 1 it was started, -1 on error.	<code>win.h</code>
<code>uk_win_wallpaper_buffer</code>	the screen-sized wallpaper to draw into → its pixels, 0	<code>win.h</code>
<code>uk_win_wallpaper_commit</code>	... drawn: shown	<code>win.h</code>
<code>uk_win_wallpaper_generate</code>	the server's own Voronoi wallpaper	<code>win.h</code>
<code>uk_win_drag_begin</code>	A drag from this	<code>win.h</code>

Name	What it does	Header
	window while the left button is held	
<code>uk_win_drag_data</code>	the last drop's payload (cap bytes) → its length	<code>win.h</code>
<code>uk_win_tray_set</code>	This program's icon (KAPI_TRAY_PX square, 0xTTRRGGBB) and tip	<code>win.h</code>
<code>uk_win_tray_clear</code>	This program's icon (KAPI_TRAY_PX square, 0xTTRRGGBB) and tip	<code>win.h</code>
<code>uk_win_tray_list</code>	for the menu bar: the icons → how many	<code>win.h</code>
<code>uk_win_tray_icon</code>	... one's pixels → 1, 0	<code>win.h</code>
<code>uk_win_tray_activate</code>	... clicked (KAPI_TRAY_OPEN, _MENU) → 1, 0	<code>win.h</code>
<code>uk_win_wheel_get</code>	lines scrolled a notch, system-wide	<code>win.h</code>
<code>uk_win_wheel_set</code>	... set (1 .. 16; the theme editor keeps it in theme.txt)	<code>win.h</code>
<code>uk_win_fullscreen_begin</code>	The display for this program alone (the server told, then the kernel's <code>kapi_fullscreen_begin</code>) → the screen's pixels (*w x *h	<code>win.h</code>
<code>uk_win_fullscreen_end</code>	The display for this program alone (the server told, then the kernel's <code>kapi_fullscreen_begin</code>) → the screen's pixels (*w x *h	<code>win.h</code>
<code>uk_win_server_info</code>	(a type)	<code>win.h</code>
<code>uk_win_server</code>	The server this program's UIKit speaks to (each server has its UIKit) → 1, 0 none runs (*out then says what the desktop would be).	<code>win.h</code>
<code>uk_shell_register</code>	"I am the shell" (the program the server started)	<code>win.h</code>
<code>uk_shell_events</code>	window opened, closed, retitled, brought to the	<code>win.h</code>

Name	What it does	Header
	front → fn	
<code>uk_shell_keys</code>	the system keys delivered to the shell first	<code>win.h</code>
<code>uk_shell_thumb</code>	window id's picture scaled to w x h into dst	<code>win.h</code>
<code>uk_shell_front</code>	window id to the front (1) / the back (0)	<code>win.h</code>
<code>uk_shell_split</code>	two windows side by side (0 0: none)	<code>win.h</code>
<code>uk_shell_dim</code>	the dim behind an overlay (0: none)	<code>win.h</code>
<code>create_window</code>	Friendly aliases used by the demos (they were <code>appkit.h</code> 's).	<code>win.h</code>
<code>present</code>	Friendly aliases used by the demos (they were <code>appkit.h</code> 's).	<code>win.h</code>
<code>Canvas</code>	(a type)	<code>canvas.h</code>
<code>UIKIT_VAR</code>	an app's background (its face)	<code>theme.h</code>
<code>uk_style_palette</code>	UK_STYLE_MILK: Milk's; any other value: CDE's	<code>theme.h</code>
<code>uk_theme_palette</code>	a named theme's colours (its own, else its style's)	<code>theme.h</code>
<code>uk_theme_load</code>	(<code>uikit::init ()</code>) calls it; idempotent)	<code>theme.h</code>
<code>uk_theme_reload</code>	read SD:/etc/theme.txt again (a new theme applied: the dock)	<code>theme.h</code>
<code>uk_theme_face</code>	the palette's shades from a face colour (a preview)	<code>theme.h</code>
<code>UkTheme</code>	(a type)	<code>theme.h</code>
<code>uk_theme_take_style</code>	A theme's style changed (a scheme chosen)	<code>theme.h</code>
<code>uk_theme_take</code>	A named theme chosen (<code>uk_themes[theme]</code>)	<code>theme.h</code>
<code>uk_theme_defaults</code>	A named theme chosen (<code>uk_themes[theme]</code>)	<code>theme.h</code>
<code>uk_theme_parse</code>	<code>theme.txt</code> 's keys over t	<code>theme.h</code>
<code>uk_theme_get</code>	the palette in use	<code>theme.h</code>

Name	What it does	Header
<code>uk_theme_set</code>	... made this one (every shade computed)	<code>theme.h</code>
<code>uk_theme_write</code>	theme.txt's text → its length	<code>theme.h</code>
<code>TextFace</code>	UIKit/text.h -- the text-rendering hook	<code>text.h</code>
<code>UIKIT_VAR</code>	(the face in use: <code>uk_textface ()</code>)	<code>text.h</code>
<code>uk_set_textface</code>	install a face; 0: back to the bitmap fonts	<code>text.h</code>
<code>uk_textface</code>	The face <code>uk_set_textface</code> installed, 0 while the bitmap fonts are in use.	<code>text.h</code>
<code>UkFaceScope</code>	Another face for the time of a scope -- a widget's own captions, a display's large digits --, the one before back at its end.	<code>text.h</code>
<code>uk_bfw</code>	The bitmap fonts' cell (the kernel's font)	<code>text.h</code>
<code>uk_bfh</code>	The bitmap fonts' cell (the kernel's font)	<code>text.h</code>
<code>uk_tw</code>	The width of s as Canvas::text draws it (no face)	<code>text.h</code>
<code>uk_tw_n</code>	The width of s as Canvas::text draws it (no face)	<code>text.h</code>
<code>uk_tpos</code>	The place (a byte index $\leq n$, a character's start) whose caret is the nearest to x px from the text's start	<code>text.h</code>
<code>uk_text</code>	s at (x, y) -- the line's top-left -- in <code>style</code> (the bitmap path)	<code>text.h</code>
<code>uk_text_clip</code>	...	<code>text.h</code>
<code>uk_text_fit</code>	Fit s into w px (a copy in out, cap bytes)	<code>text.h</code>
<code>uk_text_wrap</code>	Word-wrap the first n bytes of s into lines of w px at most, maxLines at most	<code>text.h</code>
<code>uk_text_over</code>	s at (x, y) -- the line's	<code>text.h</code>

Name	What it does	Header
	top-left -- straight over a see-through canvas (a WIN_FLAG_ALPHA window)	
<code>uk_u8_len</code>	A sequence's length from its first byte and what follows it (1 for a byte that does not start a valid sequence)	<code>text.h</code>
<code>uk_u8_get</code>	The character at s (its length in *len), a stray byte as its Latin-1 character.	<code>text.h</code>
<code>uk_u8_next</code>	The next / the previous character's start from byte i (of n).	<code>text.h</code>
<code>uk_u8_prev</code>	The next / the previous character's start from byte i (of n).	<code>text.h</code>
<code>uk_u8_put</code>	cp written as UTF-8 into o (4 bytes room) → its length.	<code>text.h</code>
<code>uk_u8_key</code>	A typed key as the text widgets store it with a face	<code>text.h</code>
<code>Label</code>	uikit/label.h -- non-interactive text on a solid background.	<code>label.h</code>
<code>Button</code>	uikit/button.h -- a push button (the theme's raised face, as a drop-down's)	<code>button.h</code>
<code>Modal</code>	uikit/dialog.h -- modal dialogs as uikit objects.	<code>dialog.h</code>
<code>MessageBox</code>	(a type)	<code>dialog.h</code>
<code>FileDialog</code>	The file dialog (<code>uk_file_open</code> / <code>uk_file_save</code> / <code>uk_folder_open</code> show it)	<code>dialog.h</code>
<code>ColorDialog</code>	Colour dialog (WPF-style <code>ColorPicker</code> dialog)	<code>dialog.h</code>
<code>PopupMenu</code>	Convenience	<code>dialog.h</code>
<code>uk_messagebox</code>	modal; buttons: <code>MB_*</code> → 1 OK / Yes, 0 Cancel / No / Esc (2: Yes-No-Cancel's No)	<code>dialog.h</code>

Name	What it does	Header
uk_file_open	The file dialogs	dialog.h
uk_file_save	with a name box (defName in it)	dialog.h
uk_folder_open	pick a directory	dialog.h
uk_color_dialog	true = OK (*color set)	dialog.h
icon_load	The picture of a file (new[])	bmp.h
bmp_decode	(the name the programs knew)	bmp.h
Calendar	uikit/calendar.h -- Calendar	calendar.h
DatePicker	(a type)	calendar.h
Checkbox	uikit/checkbox.h -- a box + check mark + label	checkbox.h
CodeEdit	(a type)	codeedit.h
ColorPicker	uikit/colorpick.h -- a colour swatch	colorpick.h
Combobox	uikit/combobox.h -- an editable Textbox with a drop-down list of suggestions (WinForms ComboBox, DropDown style)	combobox.h
DataGrid	(a type)	datagrid.h
Dropdown	uikit/dropdown.h -- a non-editable drop- down list (the sibling of Combobox, same look)	dropdown.h
uk_draw_option_list	The open list of a drop- down (Dropdown, Combobox)	dropdown.h
uk_event	a widget's event: the widget it comes from	flat.h
uk_resized	uikit/flat.h -- UIKit for a program that is not C++	flat.h
uk_chosen	a menu item chosen	flat.h
uk_window	the program's window (w x h: its inside); 0: none	flat.h
uk_window_run	the events, until the window is closed (its box, uk_window_close)	flat.h
uk_window_step	... one round of them, for a program with its	flat.h

Name	What it does	Header
	own loop: 0 once closed	
<code>uk_window_wait</code>	... one round and the wait before the next (16 ms): WHILE <code>uk_window_wait (w) ...</code> WEND	<code>flat.h</code>
<code>uk_window_close</code>	<code>uk_window_run</code> returns, <code>uk_window_step</code> gives 0	<code>flat.h</code>
<code>uk_window_width</code>		<code>flat.h</code>
<code>uk_window_height</code>		<code>flat.h</code>
<code>uk_window_min_size</code>	the smallest a resizable window goes	<code>flat.h</code>
<code>uk_window_on_resize</code>	after the user resized it: place the widgets again	<code>flat.h</code>
<code>uk_panel</code>	What holds a widget -- the <code>window</code> every maker takes -- is the window itself or a panel	<code>flat.h</code>
<code>uk_set_parent</code>	the widget taken from what holds it and put into a window or a panel	<code>flat.h</code>
<code>uk_width</code>	a widget's, a panel's, the window's size	<code>flat.h</code>
<code>uk_height</code>	What holds a widget -- the <code>window</code> every maker takes -- is the window itself or a panel	<code>flat.h</code>
<code>uk_label</code>	What holds a widget -- the <code>window</code> every maker takes -- is the window itself or a panel	<code>flat.h</code>
<code>uk_button</code>	What holds a widget -- the <code>window</code> every maker takes -- is the window itself or a panel	<code>flat.h</code>
<code>uk_textbox</code>	What holds a widget -- the <code>window</code> every maker takes -- is the window itself or a panel	<code>flat.h</code>
<code>uk_checkbox</code>	What holds a widget -- the <code>window</code> every maker takes -- is the window itself or a panel	<code>flat.h</code>
<code>uk_listbox</code>	items: "one two three"	<code>flat.h</code>
<code>uk_dropdown</code>	What holds a widget --	<code>flat.h</code>

Name	What it does	Header
	the <code>window</code> every maker takes -- is the window itself or a panel	
<code>uk_slider</code>	from 0 to max	<code>flat.h</code>
<code>uk_progress</code>	What holds a widget -- the <code>window</code> every maker takes -- is the window itself or a panel	<code>flat.h</code>
<code>uk_set_range</code>	a slider, a progress bar: its ends	<code>flat.h</code>
<code>uk_set_text</code>	A widget's text	<code>flat.h</code>
<code>uk_get_text</code>	A widget's text	<code>flat.h</code>
<code>uk_set_value</code>	Its number	<code>flat.h</code>
<code>uk_get_value</code>	Its number	<code>flat.h</code>
<code>uk_add_item</code>	a list: one more item	<code>flat.h</code>
<code>uk_clear_items</code>	a list: emptied	<code>flat.h</code>
<code>uk_item_count</code>	a list, a drop-down	<code>flat.h</code>
<code>uk_move</code>	Its number	<code>flat.h</code>
<code>uk_show</code>	Its number	<code>flat.h</code>
<code>uk_enable</code>	Its number	<code>flat.h</code>
<code>uk_shown</code>	Its number	<code>flat.h</code>
<code>uk_enabled</code>	Its number	<code>flat.h</code>
<code>uk_focus</code>	Its number	<code>flat.h</code>
<code>uk_menu_item</code>	One more item in the menu <code>title</code> (made at its first item)	<code>flat.h</code>
<code>uk_message</code>	buttons 0 OK, 1 OK Cancel, 2 Yes No, 3 Yes No Cancel → 1 OK / Yes, 0 Cancel / No (3: 2 for No)	<code>flat.h</code>
<code>uk_ask_open</code>	the file chosen ("" : none); good until the next call	<code>flat.h</code>
<code>uk_ask_save</code>		<code>flat.h</code>
Font	UIKit/font.h -- Font	<code>font.h</code>
<code>init</code>	load the theme (SD:/etc/theme.txt) and the font families; only the first call works	<code>font.h</code>
<code>font</code>	the family for <code>id</code> , or Sans if it isn't loaded	<code>font.h</code>
<code>draw_text</code>	Draw <code>s</code> into a row	<code>font.h</code>

Name	What it does	Header
	0x00RRGGBB framebuffer (W x H) via the font registry -- for app-drawn windows that don't own a uikit Canvas.	
GroupBox	uikit/groupbox.h -- GroupBox	groupbox.h
Icon	uikit/icon.h -- a magenta-keyed BMP centred above an optional label	icon.h
ImageBox	(a type)	imagebox.h
Knob	uikit/knob.h -- Knob	knob.h
uk_tr	the translation, else en itself	lang.h
uk_trc	"ctx en"'s, else en's, else en	lang.h
uk_lang_init	the system's language loaded -- before the UI is built	lang.h
uk_lang_load	that language's catalogues ("en": none) → false: none found	lang.h
uk_lang	the language in use: "en", "fr"...	lang.h
uk_lang_chosen	the system's (locale_language: "en" when none is said)	lang.h
uk_lang_choose	the system's language changed (locale_set_language; taken by the programs started next)	lang.h
VerticalStackPanel	uikit/layout.h -- automatic layout containers (Panels that re-flow their children on resize and on child add/remove, via the Widget::layout() hook).	layout.h
HorizontalStackPanel	(a type)	layout.h
UniformGridLayout	(a type)	layout.h
LcdDisplay	uikit/lcd.h -- LcdDisplay	lcd.h
ListBox	uikit/listbox.h -- ListBox	listbox.h
MenuAction	a menu item's callback	menu.h

Name	What it does	Header
	(chosen in the bar, or its shortcut typed)	
Menu	(a type)	menu.h
NumericUpDown	uikit/numeric.h -- NumericUpDown	numeric.h
uk_paint_alpha	The alpha mode	paint.h
uk_blend_px	One pixel of c at opacity a (0..255) over what the canvas holds (the alpha mode heeded).	paint.h
uk_tone	the grey profile: 128 = c, 255 = white, 0 = black	paint.h
uk_mix	t = 0 (a) .. 256 (b)	paint.h
uk_over	0..255 (0.30 R + 0.59 G + 0.11 B)	paint.h
uk_ink_on	dark or white text on c	paint.h
uk_corner	the table for radius r (clamped to 1..16), computed once and kept	paint.h
uk_rbox	fill a rounded box with a vertical gradient, top to bottom	paint.h
uk_rline	Its 1-px outline in colour c at opacity alpha.	paint.h
uk_framed	The push button in the box, from its face colour	paint.h
uk_raised	A raised face (a header, a tab, a handle, a scroll bar's thumb)	paint.h
uk_sunken	A sunken field (a text box, a list, a display)	paint.h
uk_etch_h	An etched line (a groove)	paint.h
uk_etch_v	An etched line (a groove)	paint.h
uk_etch_box	An etched rounded frame (a group box's), the same groove all round.	paint.h
uk_check_mark	The controls' marks, s x s at (x, y) (state	paint.h
uk_radio_mark	The controls' marks, s x s at (x, y) (state	paint.h

Name	What it does	Header
<code>uk_switch_mark</code>	The controls' marks, <code>s x s</code> at <code>(x, y)</code> (state)	<code>paint.h</code>
<code>uk_scroll_bar</code>	A scroll bar in the box	<code>paint.h</code>
<code>uk_slider_mark</code>	A slider's groove (the accent up to <code>fill</code> px) and its knob at <code>kx</code> (a raised rounded thumb), across <code>w x h</code> .	<code>paint.h</code>
<code>uk_progress_bar</code>	A bar that fills (a progress bar)	<code>paint.h</code>
<code>uk_popup</code>	A floating panel (a drop-down list, a menu, a drawer, a tooltip)	<code>paint.h</code>
<code>uk_corner_key</code>	Set the pixels of the box's rounded corners that are mostly outside to the magenta key.	<code>paint.h</code>
<code>uk_hilite</code>	A highlighted row (a selection, the item under the pointer)	<code>paint.h</code>
<code>uk_hilite_ink</code>	A highlighted row (a selection, the item under the pointer)	<code>paint.h</code>
<code>uk_title_strip</code>	A small window-like title strip (a dialog's)	<code>paint.h</code>
<code>uk_bead</code>	A glossy bead <code>d x d</code> at <code>(x, y)</code> in colour <code>c</code> (the Milk theme's title buttons, as OS X's)	<code>paint.h</code>
<code>uk_glyph</code>	A glyph centred on <code>(cx, cy)</code> , about <code>size</code> px across, in colour <code>c</code> .	<code>paint.h</code>
<code>uk_text_c</code>	The theme's text in a box	<code>paint.h</code>
<code>uk_text_l</code>	The theme's text in a box	<code>paint.h</code>
<code>uk_ink_for</code>	The text colour for a background	<code>paint.h</code>
<code>uk_text_w</code>	The text colour for a background	<code>paint.h</code>
<code>Panel</code>	<code>uikit/panel.h</code> -- a plain container with a solid background	<code>panel.h</code>
<code>Progress</code>	<code>uikit/progress.h</code> -- non-interactive bar, fills proportionally.	<code>progress.h</code>

Name	What it does	Header
RadioButton	UIKit/radio.h -- RadioButton	radio.h
uk_radio_checked	The checked button of group among parent's children (0 = none).	radio.h
RtStyle	Unpacked character style (public API + the "current typing style").	richtextbox.h
rt_pack	RtStyle → packed u32	richtextbox.h
rt_unpack	packed u32 → RtStyle	richtextbox.h
rt_color	palette index → 0x00RRGGBB	richtextbox.h
RichTextBox	(a type)	richtextbox.h
Scrollbar	UIKit/scrollbar.h -- draggable thumb (vertical or horizontal), value in [0,vmax].	scrollbar.h
SegmentedControl	UIKit/segmented.h -- SegmentedControl	segmented.h
uk_tint	Multiply a 0x00RRGGBB pixel by a 0x00RRGGBB tint (per channel /255).	skin.h
Skin	(a type)	skin.h
uk_decorate_window	Draw the window's frame (the modernised CDE	skin.h
uk_window_state	set the frame's state (UK_WIN_* flags) for the next uk_decorate_window	skin.h
uk_window_flags	the UK_WIN_* flags last set (0 at start)	skin.h
uk_draw_frame	A frame as uk_decorate_window draws it, into any W x H buffer (T	skin.h
Slider	UIKit/slider.h -- horizontal value in [vmin,vmax]	slider.h
Splitter	(a type)	splitter.h
HSplitter	(a type)	splitter.h
VSplitter	(a type)	splitter.h
uk_clip_ready	SystemKit is there (false: no clipboard -- the field does nothing)	sysclip.h

Name	What it does	Header
TabHost	uikit/tabhost.h -- a section that hosts several "tasks" (apps) and shows ONE at a time, chosen from a popup menu in its header (the activity-shell tab strip, po	tabhost.h
TabStrip	uikit/tabstrip.h -- TabStrip	tabstrip.h
Textarea	uikit/textarea.h -- multi-line editable text (own '\n'-separated buffer), caret-driven vertical+horizontal scroll, click to position.	textarea.h
Textbox	uikit/textbox.h -- single-line editable field	textbox.h
ToggleSwitch	uikit/toggle.h -- ToggleSwitch	toggle.h
uk_tool_glyph	draw the icon kind (WKT_*); size: 6 px at least	toolbar.h
ToolIconFn	An app's icon	toolbar.h
ToolButton	(a type)	toolbar.h
ToolBar	(a type)	toolbar.h
TreeView	uikit/treeview.h -- TreeView	treeview.h
V	uikit/vpaint.h -- shapes from their geometry, anti-aliased	vpaint.h
uk_sin	sin and cos of an angle in degrees, x 16384.	vpaint.h
uk_cos	sin and cos of an angle in degrees, x 16384.	vpaint.h
VPath	(a type)	vpaint.h
uk_q16_to_cdb	A linear level (65536 = 1.0 = 0 dBFS) → 1/100 dB (0 → -20000).	vumeter.h
VuMeter	(a type)	vumeter.h

uikit/uikit.h

uikit/uikit.h -- umbrella header for the uikit widget toolkit. Apps:

```
#include "uikit/uikit.h"          (declarations)
... build a tree of uikit::Widget subclasses, root.run() ...
and link uikit/libuikit.a         (the compiled toolkit).
```

The toolkit is a recursive, retained-mode widget tree (port of VMKernel's GUI_ELEMENT/GIMAGE): every Widget owns a Canvas; damage propagates up (valid / shouldRedraw); draw() recomposes only dirty subtrees; handleMouse() routes down with coordinate conversion; the Root adopts the kapi window canvas and presents.

(uikit/toolbar.h -- ToolBar, ToolButton, the WKT_* icons -- is included by the app itself: Letters, the Spreadsheet and Cardfile have their own ToolBar / ToolButton next to `using namespace uikit.`)

uikit/widget.h

uikit/widget.h -- Widget: the base node of the recursive tree (port of VMKernel's GUI_ELEMENT). Owns a Canvas; lives in a sibling linked-list tree. Subclasses override the virtual onDraw/onMouse/onKey. Two damage bits drive lazy recompose:

```
shouldRedraw = MY content changed (repaint me) ; valid = my subtree is current.
```

shared helpers + theme palette (ported from uikit's Ui defaults)

uk_fh: the line height widgets lay text out with; uk_fw: a character's width (a digit's with a proportional face installed -- measure real text with uk_tw / uk_text_w).

```
int uk_len (const char *s);
int uk_fw  ();
int uk_fh  ();
```

(the palette -- C_BG, C_FACE, C_TEXT, C_ACCENT, C_FIELD... -- is the theme's: uikit/theme.h)

integrated vertical scrollbar (Textarea / RichTextBox)

A widget that scrolls its own content reserves UK_SBW px on its right edge and shows a draggable thumb there only when the content overflows. uk_thumb computes the thumb rect; uk_thumb_pos is the drag inverse; uk_draw_vscroll paints it.

```
static const int UK_SBW = 10;          // reserved right-edge gutter width
```

```
struct UkThumb { bool show; int y, h; }; // thumb top/height within a track of trackH px
```

The thumb of a trackH px track for total units of content, view of them visible, scrolled to pos, with show false when all fits (or the track is 6 px or less).

```
UkThumb uk_thumb (long total, long view, long pos, int trackH);
```

A cursor offset cy within the track → scroll pos in [0, total-view] (thumb centred).

```
long uk_thumb_pos (int cy, int trackH, long total, long view, int thumbH);
```

Paint the bar (a groove in a shade of bg, the thumb a raised pill) at the right-edge gutter (x,y, w x trackH); hot: pointed / dragged.

```
void uk_draw_vscroll (Canvas &cv, int x, int y, int w, int trackH, const UkThumb &t,
unsigned bg, bool hot = false);
```

A list's own scroll bar under the mouse (a widget that draws uk_draw_vscroll): its thumb dragged, a press on its groove a page up / down. The bar: barW px from barX, its track trackH px from ty; total / view / *pos in the list's units (rows, px). Called from onMouse before the rows are looked at: true = the bar took the event (*pos may have moved: repaint, and leave the rows alone). The thumb held goes on following the pointer outside the bar (mx < 0: the pointer left, it is let go).

```
struct UkBarDrag
{
    bool held, wasDown;
    UkBarDrag () : held (false), wasDown (false) {}
    bool mouse (int mx, int my, int bl, int barX, int barW, int ty, int trackH, long
total, long view, long *pos)
    {
        bool down = bl != 0, press = down && !wasDown;
        wasDown = down;
        if (held)
        {
            if (!down || mx < 0) held = false;
            else { UkThumb t = uk_thumb (total, view, *pos, trackH); *pos =
uk_thumb_pos (my - ty, trackH, total, view, t.h); }
            return true;
        }
        if (total <= view || mx < barX || mx >= barX + barW || my < ty || my >= ty +
trackH) return false;
        if (press)
        {
            UkThumb t = uk_thumb (total, view, *pos, trackH);
            int cy = my - ty;
            if (cy >= t.y && cy < t.y + t.h) held = true;
            else
            {
                long p = *pos + (cy < t.y ? -view : view), most = total - view;
                *pos = p < 0 ? 0 : p > most ? most : p;
            }
        }
        return true;
    }
};
```

The pointer's shape (kapi v81, KAPI_CURSOR_*). A widget asks for its shape from its onMouse, each time the pointer moves over it: uk_cursor (KAPI_CURSOR_TEXT) in a text field, _HAND over a link, _SIZE_H on a column's edge... Root::ptrEvent (root.cpp) starts each pointer event with the arrow and tells the kernel when what the widgets asked for has changed -- so a widget that asks nothing shows the arrow, and nobody has to put it back.

```
void uk_cursor (int shape);
```

```
class Widget;
class RadioButton;
typedef void (*Action) (Widget &); // fired on click/toggle/change; gets the
widget
```

Anchors (WinForms-style): which parent edges a child keeps a constant distance to as the parent resizes. The DEFAULT container layout() honours these so a child resizes / repositions passively (the app never recomputes geometry on resize):

LEFT+RIGHT	-> width stretches		LEFT only	-> pinned left (default, fixed w)
TOP+BOTTOM	-> height stretches		RIGHT only	-> rides the right edge

(reposition)

FILL -> stretches both | BOTTOM only-> rides the bottom edge (reposition)

Resizing an anchored child cascades into ITS layout() (e.g. a grid re-flows its keys).

```
enum {
    ANCHOR_LEFT = 1, ANCHOR_TOP = 2, ANCHOR_RIGHT = 4, ANCHOR_BOTTOM = 8,
    ANCHOR_FILL = ANCHOR_LEFT | ANCHOR_TOP | ANCHOR_RIGHT | ANCHOR_BOTTOM
};

class Widget
{
public:
    Canvas canvas;
    int left, top, width, height; // geometry RELATIVE to the parent
    int scrollX, scrollY; // content offset (scrollview)
    int tag; // app/dialog-defined id (e.g. dialog button result)
    int colSpan, rowSpan; // cells spanned in a UniformGridLayout (default 1)
    int anchor; // ANCHOR_* mask (default LEFT|TOP = pinned top-left)
    int lytW, lytH; // my size at the last layout (for anchor deltas)
    bool transparent; // blit me with the magenta key?
    bool valid; // subtree composited & current (IsValid)
    bool shouldRedraw; // my own content needs repaint (ShouldRedraw)
    bool hidden; // skip me in composite + hit-test (e.g. an inactive
tab)
    bool hasFocus, canFocus, catchOutside;
    bool modal; // a modal child captures ALL of the parent's input
    bool disabled, hover, pressed; // interaction state (widgets use these)
    const char *tip; // tooltip text (0 = none): Root shows it after a
hover pause

    Widget *parent, *firstChild, *lastChild, *prevSib, *nextSib;
    Widget *prevHandled; // last child that took the mouse (for mouse-
leave)
    // The reserve (uikit is a shared library: uikit/abi.h). Programs allocate and
derive widgets with
    // the layout they were built with: a field added by a later version of the
library goes HERE
    // (or behind `ext`, which the library may allocate) -- never in the middle of a
class, never
    // at the end of a derived one.
    void *ext = 0;
    unsigned long reserved_[4] = { 0, 0, 0, 0 };

    Widget (int l, int t, int w, int h);
    virtual ~Widget ();

    // ---- damage -----
    void invalidate (bool redraw); // redraw=true: repaint me; false: just
recompose up
    virtual void resizeTo (int w, int h); // virtual: a surface-backed view
overrides to pin its size
    void setBounds (int w, int h); // resize the LOGICAL size (no canvas
realloc) + relayout;
    // for surface-backed widgets (adopted over-allocated buffer)

    // ---- tree -----
    void addChild (Widget *c);
    void removeChild (Widget *c);
    void bringToFront (); // move me last in z-order (topmost)
```

```

// ---- recursive composite + event routing -----
void draw (); // recompose this subtree's canvas (lazy)
bool handleMouse (int mx, int my, int bl, int br, int bm, int wheel); //
(mx,my) local
bool handleKey (long k);

// ---- focus -----
void clearFocusTree ();
void focusPathUp ();
void setFocus (); // focus me (clear the tree, light up my path)
// Tab / Shift+Tab (back): the focus to the next / previous control of this
subtree, in the
// order they were added (depth first; hidden or disabled ones skipped, the ends
wrap). fields:
// only the text fields (isField). Done by handleKey when no one takes the Tab:
in a dialog
// (a modal) every control; in a window, from a text field to the next text
field.
bool tabFocus (bool back, bool fields);

// ---- overridables -----
virtual void onDraw () {} // paint own content into `canvas`
virtual bool onMouse (int, int, int, int, int, int) { return false; }
virtual bool onKey (long) { return false; }
virtual RadioButton *asRadio () { return 0; } // (no RTTI) a RadioButton says
so
virtual bool isField () { return false; } // a text field (Textbox and kin): a
Tab stop in a window
virtual void onTabFocus () {} // focused by Tab (a field: its caret at
the end)
// The colour behind this widget (its parent's background): what its rounded,
anti-aliased
// corners blend into. A container with its own background returns it.
virtual unsigned bgColor () { return parent ? parent->bgColor () : C_BG; }
// Reposition/resize children on resize / child add/remove. The DEFAULT applies
the
// children's ANCHOR_* (passive resize); layout containers (Splitter, StackPanel,
// UniformGridLayout, TabHost) override with explicit placement. Never call
directly.
virtual void layout ();
// The reserve of virtual functions (uikit/abi.h): a program's vtables are built
by its compiler
// with the slots of the headers it was built with. A virtual added to Widget by
a later version
// of the library TAKES ONE OF THESE (renamed, same place): an older program's
widgets then
// answer with this empty default. Never add a virtual anywhere else in an
exposed class.
virtual void uk_reserved0 () {}
virtual void uk_reserved1 () {}
virtual void uk_reserved2 () {}
virtual void uk_reserved3 () {}
virtual void uk_reserved4 () {}
virtual void uk_reserved5 () {}
virtual void uk_reserved6 () {}
virtual void uk_reserved7 () {}
};

```

uikit/root.h

uikit/root.h -- the top widget, bound to the kapi WINDOW canvas. Runs the event loop: feeds the kapi pointer/key streams into handleMouse/handleKey, and recomposes + uk_win_present()s only when the tree is dirty (valid==false).

The loop's three steps, the same for a window and for an applet (a Control Panel applet: applet_proto.h -- the app started with "--applet " draws into the host's pane instead of a window of its own; a Root made then adopts that surface). Root::run and the modal dialogs go through them; an app with its own loop should too.

```
bool uk_applet ();           // running as an applet? (its arguments said so)
void uk_pump ();             // the events: the window's (pump_events), an
                             // applet's host's
void uk_present ();          // what was drawn shown: uk_win_present, or told to the
                             // host
bool uk_quit ();             // time to end: the close box, or the host's AP_CLOSE
                             // its end
bool uk_applet_send (int type, const void *data = 0, unsigned len = 0); // a message
                             // to the host
```

The host's messages other than AP_PTR / AP_KEY / AP_CLOSE (an applet's own protocol: Jet's web view, Apps/jet/webview_proto.h) given to fn (data NUL-terminated after its len bytes), from uk_pump. A host other than the Control Panel names its IPC service: "--applet " (the applet ends when that service is gone).

```
void uk_applet_on_message (void (*fn) (int type, const void *data, int len));
```

(v94) A program's other windows (docs/MULTI-WINDOW-STUDY.md): a Root made with NewWindow is one more window of the program (uk_win_new, uikit/win.h), beside the first Root (its main window). The first Root's run () / step () serves every one: their events (routed by the window they are for), their onTick, their drawing. Its close box asks onClose () (the default: closeWindow ()); the program's end is still the first window's close.

```
struct NewWindow {};
```

(v95) The program's icon in the menu bar's status area: a picture file (an app's icon.bmp: brought to KAPI_TRAY_PX square, its magenta see-through) and a tip → true; false (an older system, no picture). A double click on it shows the program's first window again (even minimised) and calls its onTray (KAPI_TRAY_OPEN); a right click onTray (KAPI_TRAY_MENU). uk_tray_clear: taken away (the program's end does it too).

```
bool uk_tray (const char *picture, const char *tip);
void uk_tray_clear ();
```

```
class Root : public Widget
```

```
{
public:
    unsigned bg;           // client-area background colour
    Root (int w, int h, const char *title);           // decorated window
    Root (int x, int y, int w, int h, const char *title, unsigned flags); //
    positioned / borderless
    // (v94) Another window of the program (x, y negative: placed by the system). If
    // it cannot be
    // made (no graphics server's windows left, no memory), win0opened () says false
    // (nothing is drawn).
    Root (NewWindow, int x, int y, int w, int h, const char *title, unsigned flags =
0);
    int winNumber () const { return (int) m_reserved[0]; } // its number in the
```

```

program (0: the first)
    bool winOpened () const { return m_reserved[1] == 0; } // (false: not made, or
closed)
    void closeWindow (); // this window closed (not the first: the program
ends then)
    void winSelect (); // the window calls (uk_win_*) act on this window
(uk_win_select)
    static Root *winFirst (); // the program's first window
    static int winCount (); // how many windows are open
    static void paintAll (); // every window that changed drawn and shown
    void setBg (unsigned c) { bg = c; invalidate (true); }
    unsigned bgColor () override { return bg; }
    void onDraw () override;
    void run ();
    bool step (); // one round of run (): false once it is time to end
(attach () first)
    void attach (); // hook the kapi pointer / key streams (run () does
it); for
// an app that pumps its own loop (pump_events + draw +
present)
    virtual void onTick () {} // called once per run() loop (~60 Hz): polling,
timers

// Drag & drop (ABI v42). onDrop: something was dropped at (x,y) (client coords)
--
// type DND_TEXT / DND_FILES ('\n'-separated paths), data NUL-terminated, flags
// DND_F_COPY (Ctrl held). onDragOver: a drag hovers (x,y); leave = it went away
// (highlight a drop target). onDragDone: our own drag (uk_win_drag_begin) ended
--
// targetPid (0 = none), flags DND_F_COPY / DND_F_CANCEL / DND_F_DESKTOP.
virtual void onDrop (int x, int y, int type, const char *data, int len, unsigned
flags)
{ (void) x; (void) y; (void) type; (void) data; (void) len; (void) flags; }
virtual void onDragOver (int x, int y, bool leave, unsigned flags) { (void) x;
(void) y; (void) leave; (void) flags; }
virtual void onDragDone (int targetPid, unsigned flags) { (void) targetPid;
(void) flags; }

static Root *current (); // the active window (for modal dialogs)

// The window's frame (kapi v64). setResizable (true): the app lays itself out at
any size
// (its anchors, its layout do) -- then its maximise button works: the window
fills the
// work area (between the menu bar and the dock) and back; onResized () follows
(the new
// size: width, height). The window menu (its button, top left): Restore /
Maximise,
// Minimise, Close.
void setResizable (bool on);
// The smallest client area the frame can be dragged to (kapi v82). Without it:
half the size
// the window has when it becomes resizable (160 x 100 at least). Before or after
setResizable.
void setMinSize (int w, int h);
bool resizable () const { return m_resizable; }
bool maximised () const { return m_maxed; }
void maximise (bool on);
// A resizable window taller (or wider) than the work area: shrunk to it and
moved into it

```

```

    // (the dock no longer over its bottom) -- at the start, once its children are
    anchored.
    void fitWorkArea ();
    virtual void onResized () {}
    // The screen's size changed (GUI_EVENT_DISPLAY_RESIZE, kernel v66: the Control
    Panel's Display
    // applet): an app placed by the screen's size (the menu bar, the dock...) places
    itself again
    // here. Then, ~0.3 s later (the dock moved: the work area is the new one), run
    () fits the
    // window: maximised, to the whole work area again (the size it goes back to kept
    inside it);
    // else a window past the work area is moved into it, shrunk if it is resizable.
    virtual void onDisplayResize (int w, int h) { (void) w; (void) h; }
    // The reserve of virtual functions for the window (uikit/abi.h; Widget's own
    come before): a
    // virtual added to Root by a later version of the library takes one of these.
    virtual void onClose ();          // (v94) its close box (not the first window's):
    closeWindow ()
    void uk_rootReserved0 ();          // (the slots' former names: entries of the
    table, never removed)
    void uk_rootReserved1 ();
    // (v95) A click on the program's icon of the status area (uk_tray):
    KAPI_TRAY_OPEN (a double click:
    // the first window already shown again), KAPI_TRAY_MENU (a right click). To the
    first window.
    virtual void onTray (int kind);
    virtual void uk_rootReserved2 () {}
    virtual void uk_rootReserved3 () {}
    virtual void uk_rootReserved4 () {}
    virtual void uk_rootReserved5 () {}
    virtual void uk_rootReserved6 () {}
    virtual void uk_rootReserved7 () {}
    void windowMenu ();

    // Tooltips (Widget::tip): after the pointer rests ~0.6 s over a widget with a
    tip,
    // a small box shows the text next to it; any pointer event hides it.
    void tooltipTick ();
    void tooltipHide ();

private:
    Widget *m_tipBox;          // the shown tooltip (a child), or 0
    int m_mx, m_my;            // last pointer position (client coords)
    unsigned m_moveT;          // ticks of the last pointer event
    bool m_tipDone;            // already shown for this rest
    bool m_resizable, m_maxed;  // (setResizable, maximise)
    int m_minW = 0, m_minH = 0; // (setMinSize; 0: chosen by setResizable)
    void frameResize (int x, int y, int cw, int ch); // the frame was dragged
    (GUI_EVENT_WINRESIZE)
    int m_rx, m_ry, m_rw, m_rh; // the window's place and size before it was
    maximised
    bool m_dispPending;        // (GUI_EVENT_DISPLAY_RESIZE: displayTick fits
    the window)
    unsigned m_winFlags;        // (its WIN_FLAG_*: a borderless one places
    itself)
    unsigned m_dispt;
    void displayTick ();
    void init (unsigned *fb);    // shared ctor tail (adopt canvas + decorate +
    register)

```

```

    void initApplet ();           // ... an applet's: the host's surface, no window
    static Root *&active ();      // single active window per app (reachable from C
callbacks)
    void *m_ext = 0;             // the reserve (UIKit/abi.h): the window's later
fields
    unsigned long m_reserved[4] = { 0, 0, 0, 0 }; // [0] the window's number (v94),
[1] 1: not opened / closed
public:
    static void ptrEvent (unsigned long, int ev, gui_value v); // (the kernel's
event streams; an
    static void keyEvent (unsigned long, int ev, gui_value v); // applet's host's,
re-packed alike)
};

```

UIKit/win.h

UIKit/win.h -- UIKit's window API: a program's windows and what it asks of the graphics server, as plain C functions (uk_win_*, C linkage: entries of SD:/lib/UIKit.so; a C program includes this header alone and links lib/UIKit.imp_c.a, a C++ one has it through UIKit/UIKit.h and links lib/UIKit.imp.a).

Until 2026-10-08 these were AppKit's kapi_* window calls (kapi_create_window, kapi_present, kapi_set_menu, kapi_win_list...). The graphics server is a program of its own (Elegant on the desktop; PocketUI, the pocket and console modes' server, to come -- docs/POCKETUI-TECH-STUDY.md), and each server has its UIKit, loaded under the name SD:/lib/UIKit.so: UIKit's port (UIKit/port/) is the only code that speaks its server's protocol (Elegant's: UIKit/port/elegant.h). So the window API is UIKit's, and a program's binary runs on every server. AppKit keeps the kernel's side: the transport (kapi_ws_ctl), the event pump (kapi_pump_*, kapi_should_exit), the screen (kapi_screen_size), the full screen's kernel primitives (kapi_present_fb, kapi_fullscreen_direct).

The renaming, for a program written with the old names: kapi_create_window → uk_win_create, and every other window call kapi_ → uk_win_ without a doubled "win_" (kapi_win_list → uk_win_list), with these: kapi_move_window → uk_win_move, kapi_resize_window(2) → uk_win_resize(2), kapi_set_window_alpha → uk_win_alpha, kapi_get_chrome → uk_win_chrome, kapi_set_key / click / pointer_handler → uk_win_on_key / _on_click / _on_pointer, kapi_set_cursor → uk_win_cursor, kapi_set_menu / get_menu → uk_win_menu_set / _menu_get, kapi_win_move → uk_win_place, kapi_win_desk → uk_win_to_desk, kapi_list_windows → uk_win_apps, kapi_raise_app / toggle_app → uk_win_app_raise / _app_toggle, kapi_get / set_wheel_speed → uk_win_wheel_get / _wheel_set, kapi_fullscreen_begin / _end → uk_win_fullscreen_begin / _end.

The window flags (WIN_FLAG_), the events (GUI_EVENT_, GUI_PTR_, KEY_) and the structures (struct kapi_chrome, kapi_win_geom, kapi_win_info, kapi_tray_info) keep their names and their definitions (appkit/appkit.h, kern/kapi_abi.h): the events still come through the kernel's pump.

A program's windows: its first (0, made by uk_win_create / _ex) and others (uk_win_new); the calls below that act on "this window" act on the one uk_win_select chose (0 until then). What each window was asked is kept by UIKit and asked again of a graphics server started again (a crash, a restart): the program does nothing.

MIT License -- Copyright (c) 2026 Stéphane Wegener and the Onyx contributors. Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions: The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software. THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED.

the program's windows

The program's first window, a client area of $w \times h$ pixels titled `title`, placed by the server → its canvas (0x00RRGGBB pixels, w a row), 0 on failure (no server, bigger than the screen, no memory).

```
unsigned *uk_win_create (int w, int h, const char *title);
```

The program's first window at x, y (its frame's top left; negative: placed by the server) with the flags (WIN_FLAG_*) → its canvas, 0 on failure.

```
unsigned *uk_win_create_ex (int x, int y, int w, int h, const char *title, unsigned flags);
```

Another window of the program (after its first: 1 .. KAPI_WS_WINDOWS_MORE of them), as `uk_win_create_ex` → its number (> 0) and its canvas in `*canvas`, -1 (no room, no server). The calls act on it once `uk_win_select` chose it.

```
int uk_win_new (int x, int y, int w, int h, const char *title, unsigned flags, unsigned **canvas);
```

The window the calls act on (0: the first; -1: only asked) → the one they acted on until now, -1 no such window.

```
int uk_win_select (int win);  
void uk_win_destroy (int win); // window win (> 0) closed and forgotten (the first: the program's end)
```

this window

```
void uk_win_move (int x, int y); // its frame moved to x, y (screen coordinates)
```

Its client area set to $w \times h$, within the canvas it was created with (which stays) → the canvas, 0 no window.

```
unsigned *uk_win_resize (int w, int h);
```

As `uk_win_resize`, but the canvas and the frame's copies grow past their first size when needed (new memory at the same addresses: their pixels are lost -- redraw them; the frame: `uk_win_chrome` again) → the canvas, `*stride` its pixels a row; 0 (no memory: the size kept).

```
unsigned *uk_win_resize2 (int w, int h, int *stride);
```

on $!= 0$: its edges and corners can be dragged, its client area never under $\text{min_w} \times \text{min_h}$ → 0, -1 (no window, a borderless or fixed one). At the release its pointer handler gets GUI_EVENT_WINRESIZE: the program resizes itself.

```
int uk_win_resizable (int on, int min_w, int min_h);  
void uk_win_alpha (int alpha); // its opacity, 0 .. 255 (fades)
```

Its frame's place and size, its client area's size, the work area (the screen less the menu bar and the dock) and its KAPI_WIN_* state into `*out` → 0, -1 no window.

```
int uk_win_geometry (struct kapi_win_geom *out);
```

Its surfaces for the frame UIKit draws (`uikit/skin.cpp`): the client canvas, the frame's active and inactive copies (0: borderless), their size, the insets, the title → 1, 0 no window.

```
int uk_win_chrome (struct kapi_chrome *out);  
void uk_win_present (void); // its pixels changed: the server shows them
```

One line of text in the kernel's bitmap font at x, y of its canvas, colour c (0x00RRGGBB), the background kept.

```
void uk_win_draw_text (int x, int y, const char *s, unsigned c);
void uk_win_on_key (gui_handler fn); // GUI_EVENT_KEY to fn (value: the
character or KEY_*)
void uk_win_on_click (gui_handler fn); // GUI_EVENT_CANVAS_CLICK / _MOTION to fn
(the old click handler)
void uk_win_on_pointer (gui_handler fn); // the whole pointer stream to fn
(GUI_EVENT_PTR_*, drops, WINCTL, WINRESIZE)
```

The pointer's shape over its client area (KAPI_CURSOR_*), kept until changed → the shape it had, -1.

```
int uk_win_cursor (int shape);
void uk_win_cursor_pos (int *x, int *y); // the pointer, relative to its client
area
int uk_win_cursor_shown (void); // 1 the pointer is drawn, 0 hidden (keyboard
only), -1 not known
```

the menu bar

This program's menus (the first window's) and the handler that gets GUI_EVENT_MENU (value: the item's id). The spec, '\n'-separated: "M" a menu, "\t\t" an item, "-" a separator (programs normally use uikit::Menu) → 1, 0.

```
int uk_win_menu_set (const char *spec, gui_handler h);
```

For the menu bar: the active window's spec and title → a serial that changes with them (0: none).

```
unsigned uk_win_menu_get (char *buf, unsigned cap, char *title, unsigned tcap);
int uk_win_menu_command (int id); // item id sent to the active window (MENU_QUIT:
asked to close) -> 1 delivered
```

every window: the shell's, the remote desktop's

```
int uk_win_list (struct kapi_win_info *out, int max); // the windows, bottom to top
(at most max) -> how many
int uk_win_raise (unsigned id); // window id to the front (it gets the keys) ->
0, -1
int uk_win_close (unsigned id); // window id asked to close (as its close box) ->
0, -1
int uk_win_minimise (unsigned id); // window id (0: the caller's) minimised until
raised -> 0, -1
int uk_win_place (unsigned id, int x, int y); // window id's frame moved to x, y ->
0, -1
```

Window id (0: the caller's) to workspace n (-1: every one; -2: only asked) → its workspace (-1: all), -3 no window.

```
int uk_win_to_desk (unsigned id, int n);
```

The workspaces: set >= 0 shows that one, count > 0 sets how many (1 .. KAPI_DESK_MAX); -1 / 0 keep them → the current one | the count << 8 | a counter bumped at every change << 16.

```
int uk_win_desk (int set, int count);
```

The pixels of a rectangle of window id's client area (part 0) or of its frame (1 active, 2 inactive) into dst (stride pixels a row), clipped to it → 0, -1. KAPI_WIN_DESKTOP: the desktop, read whole.

```
int uk_win_read (unsigned id, int part, int x, int y, int w, int h, unsigned *dst,
int stride);
```

The names of the open apps (a window on the current workspace, not a WIN_FLAG_SYSTEM one), one a line → how many.

```
int uk_win_apps (char *b, unsigned s);
int uk_win_app_raise (const char *name); // app name's window to the front -> 1, 0
not running / no window
```

App name toggled → 0 it was running and is asked to close, 1 it was started, -1 on error.

```
int uk_win_app_toggle (const char *name);
```

the wallpaper

```
unsigned *uk_win_wallpaper_buffer (int *w, int *h); // the screen-sized wallpaper to
draw into -> its pixels, 0
void uk_win_wallpaper_commit (void); // ... drawn: shown
int uk_win_wallpaper_generate (unsigned base, int pts, unsigned seed); // the
server's own Voronoi wallpaper
```

drag and drop

A drag from this window while the left button is held: type DND_TEXT / DND_FILES ('\n'-separated paths), len bytes (4 KB at most), a label on the pointer → 1 started. The targets get GUI_EVENT_DRAG_OVER / _DROP, the source GUI_EVENT_DRAG_DONE (to the pointer handler).

```
int uk_win_drag_begin (int type, const void *data, unsigned len, const char *label);
int uk_win_drag_data (int *type, void *buf, unsigned cap); // the last drop's
payload (cap bytes) -> its length
```

the status area (the menu bar's tray)

This program's icon (KAPI_TRAY_PX square, 0xTTRRGGBB) and tip; h gets GUI_EVENT_TRAY (KAPI_TRAY_OPEN, _MENU) → 1, 0. (uikit::uk_tray does it from a picture.)

```
int uk_win_tray_set (const unsigned *px, const char *tip, gui_handler h);
void uk_win_tray_clear (void);
int uk_win_tray_list (struct kapi_tray_info *out, int max); // for the menu bar: the
icons -> how many
int uk_win_tray_icon (unsigned pid, unsigned *px); // ... one's pixels -> 1, 0
int uk_win_tray_activate (unsigned pid, int kind); // ... clicked
(KAPI_TRAY_OPEN, _MENU) -> 1, 0
```

the wheel

```
int uk_win_wheel_get (void); // lines scrolled a notch, system-wide
void uk_win_wheel_set (int lines); // ... set (1 .. 16; the theme editor keeps it in
theme.txt)
```

the full screen

The display for this program alone (the server told, then the kernel's kapi_fullscreen_begin) → the screen's pixels (*w x *h; kapi_present_fb shows them), 0 refused. uk_win_fullscreen_end gives it back.

```
unsigned *uk_win_fullscreen_begin (int *w, int *h);
void uk_win_fullscreen_end (void);
```

the graphics server

```
#define UK_MODE_DESKTOP      0    // uk_win_server's mode: Elegant's desktop
#define UK_MODE_POCKET      1    // PocketUI's pocket mode
#define UK_MODE_CONSOLE     2    // PocketUI's console mode
#define UK_SC_REGULAR       0    // size classes: the desktop
#define UK_SC_COMPACT       1    // a small landscape screen
#define UK_SC_NARROW        2    // a portrait screen
#define UK_SC_CONSOLE       3    // a television, a pad
struct uk_win_server_info
{
    unsigned size;           // set by the caller: sizeof (struct uk_win_server_info)
    // (it may grow at its end)
    char name[16];          // the server's name ("elegant")
    int mode;               // UK_MODE_*
    int screen_w, screen_h; // the screen, in the windows' pixels
    int work_x, work_y, work_w, work_h; // the work area: where a filled window
    // goes
    int scale;              // the composition's scale, in percent (100: none)
    int size_class;         // UK_SC_*
    int reserved[8];
};
```

The server this program's UIKit speaks to (each server has its UIKit) → 1, 0 none runs (*out then says what the desktop would be).

```
int uk_win_server (struct uk_win_server_info *out);
```

the shell's calls (PocketUI's: the pocket and console shells)

The desktop's UIKit answers -KAPI_ENOSYS to each: Elegant's shell programs (menubar, dock) use the calls above. (Their meaning is PocketUI's, docs/POCKETUI-TECH-STUDY.md section 4.3: written with it, at its phase P5.)

```
int uk_shell_register (void);           // "I am the shell" (the program the
server started)
int uk_shell_events (gui_handler fn);   // window opened, closed, retitled,
brought to the front -> fn
int uk_shell_keys (const int *keys, int count); // the system keys delivered to
the shell first
int uk_shell_thumb (unsigned id, unsigned *dst, int w, int h); // window id's
picture scaled to w x h into dst
int uk_shell_front (unsigned id, int front); // window id to the front (1) /
the back (0)
int uk_shell_split (unsigned left, unsigned right); // two windows side by side (0 0:
none)
int uk_shell_dim (int alpha);           // the dim behind an overlay (0: none)

}
#endif
```

Friendly aliases used by the demos (they were appkit.h's).

```
unsigned *create_window (int w, int h, const char *t);
void present (void);
```

uikit/canvas.h

uikit/canvas.h -- Canvas: a 0x00RRGGBB pixel buffer with blit/fill/text (port of VMKernel's GIMAGE). Each Widget owns one. It either OWNS its pixels (alloc) or borrows them (adopt, e.g. the window canvas from uk_win_create).

stride is the buffer's real row width in pixels; w is the LOGICAL (visible) width. They differ when a Canvas shows a sub-rect of an over-allocated buffer -- e.g. a shell surface allocated at screen width but displayed at the (smaller, resizable) viewport size. All blits step rows by stride and clamp to w.

```
#define UK_TRANSPARENT_KEY 0x00FF00FFu    // magenta = skipped on a transparent
blit

class Font;    // uikit/font.h -- bitmap glyph table (drawFont)

class Canvas
{
public:
    unsigned *px; int w, h, stride; bool owns;
    int capH;    // allocated rows (stride = allocated row width)
    // The reserve (uikit is a shared library: uikit/abi.h): a program allocates a
    Canvas with the size
    // it was built with, so a later version of the library can only use what is kept
    here.
    void *ext = 0;
    unsigned long reserved_[2] = { 0, 0 };

    Canvas ();
    ~Canvas ();

    void release ();
    bool alloc (int W, int H);    // own a fresh WxH buffer (stride = W)
    void adopt (unsigned *p, int W, int H);    // borrow a buffer, stride = W
    (window canvas)
    void adopt (unsigned *p, int W, int H, int Stride);    // borrow a sub-rect of a
    wider buffer
    // Grow-only: reuses the buffer when W<=stride && H<=capH (just changes the
    logical
    // size); otherwise reallocates to ~2x (amortised). So a resizing widget (a key
    in a
    // reflowing grid) rarely reallocates. Content is lost on a real grow (caller
    repaints).
    bool resize (int W, int H);
    void setLogical (int W, int H);    // change the visible w/h only
    (buffer/stride kept)

    void clear (unsigned c);
    void pixel (int x, int y, unsigned c);
    void fillRect (int x, int y, int rw, int rh, unsigned c);
    void frameRect (int x, int y, int rw, int rh, unsigned c);
    void text (int x, int y, const char *s, unsigned c);
    // Draw `s` from a loaded bitmap Font at (x,y), top-left, scaled by `scale`
    (>=1), in
    // `style` (0=reg 1=italic 2=bold 3=bold+italic). Only ink pixels are written
    (the
    // background shows through). Implemented in font.cpp.
    void drawFont (int x, int y, const char *s, const Font &f, unsigned color, int
    scale = 1, int style = 0);
    // Blit src into this canvas at (dx,dy); if transparent, magenta pixels are
    skipped.
    // Clipped to this canvas -> the parent only takes what fits (free clipping).
    void putOther (const Canvas &src, int dx, int dy, bool transparent);
};
```

uikit/theme.h

uikit/theme.h -- the desktop's look (docs/gui-redesign/README.md, "a modernised CDE"): the palette every widget and every window frame draws with. A theme gives ONE colour per kind of window frame (active: the window in front; inactive: the others) and the apps' face; every other shade is computed from those (uikit/paint.h: the grey profile). Read once by uikit::init () from SD:/etc/theme.txt:

```
    theme      = Peach          a named theme. Two kinds: CDE's framed title buttons --
Peach, Steel,
                                Sage, Brick, Slate (the frame of the window in front; the
ones
                                behind: Grey) --, or OS X's coloured beads, the frame
melting into
                                the window (no line between them) -- Milk (soft greys)
and Dark
                                Coffee (black coffee's browns, a caramel accent)
    style      = cde            the frames' look, if not the theme's: cde or milk
    active     = 0xF0B07A       or the colours themselves (they override the theme's)
    inactive   = 0xACACB0
    window     = 0xD0C2BA       the windows' content: the apps' face, their background
("face"
    button     = 0xD0C2BA       before: still read)
                                the push buttons, the drop-downs' faces (default: the
window's)
    field      = 0xF6F3F1       the text fields and the lists (default: the window's,
nearly white)
    accent     = 0x4992A7       focus, selection, checks
    outline    = dark           the frames' 1-px outline: none, dark or black
    menubar    = 0xD0C2BA       the menu bar (default: the window's)
    dock       = 0xA4BACE       the dock's face
```

Without the file (or a key), the defaults below: Peach over CDE's beige, a teal accent; a colour left out takes the style's own (uk_style_palette: Milk's greys, its Aqua blue...). Each text colour is black or white by the brightness of what it is written on.

The palette (0x00RRGGBB), set by uk_theme_load (); the names are the toolkit's since its start.

```
UIKIT_VAR (unsigned, C_BG);           // an app's background (its face)
UIKIT_VAR (unsigned, C_FACE);          // a control's face (a button, a handle)
UIKIT_VAR (unsigned, C_FACE_HI);       // ... under the pointer
UIKIT_VAR (unsigned, C_FACE_DN);       // ... pressed
UIKIT_VAR (unsigned, C_BORDER);        // a control's outline
UIKIT_VAR (unsigned, C_TEXT);          // text on the face
UIKIT_VAR (unsigned, C_ACCENT);        // focus, selection, checks, carets
UIKIT_VAR (unsigned, C_DIS);           // disabled text
UIKIT_VAR (unsigned, C_FIELD);         // a text field's, a list's background
UIKIT_VAR (unsigned, C_FIELD_TEXT);    // text in a field
UIKIT_VAR (unsigned, C_SEL_TEXT);      // text on the accent (a selected row)
UIKIT_VAR (unsigned, C_FRAME_ACTIVE);  // the window in front's frame
UIKIT_VAR (unsigned, C_FRAME_INACTIVE); // the other windows' frames
UIKIT_VAR (int, UK_OUTLINE);           // the frames' 1-px outline: 0 none, 1 dark, 2
black
UIKIT_VAR (unsigned, C_DOCK);          // the dock's face (and the drawers' frame)
UIKIT_VAR (unsigned, C_BUTTON);        // a push button's / a drop-down's face (the
window's by default)
UIKIT_VAR (unsigned, C_BUTTON_TEXT);   // ... its text
UIKIT_VAR (unsigned, C_MENUBAR);       // the menu bar's face (the window's by
default)
```

The frames' look: CDE's (framed title buttons) or Milk's (Xfce's Milk theme, as OS X: the title buttons coloured beads -- close red, minimise amber, maximise green --, the title's gradient down to the window's colour, C_BG, the borders that colour: the frame melts into the window).

```
enum { UK_STYLE_CDE = 0, UK_STYLE_MILK = 1 };
UIKIT_VAR (int, UK_STYLE);
```

A style's own colours: what a theme of it takes when theme.txt does not say. (button, field: 0xFF000000 = from the window's, UK_AUTO below; outline: 0 none, 1 dark, 2 black.)

```
struct UkPalette { unsigned face, accent, inactive, dock, button, field; int outline;
};
const UkPalette &uk_style_palette (int style); // UK_STYLE_MILK: Milk's; any other
value: CDE's
```

The named themes (the active frame's colour, the frames' style; pal: a theme with colours of its own -- Dark Coffee --, 0: its style's).

```
struct UkNamedTheme { const char *name; unsigned frame; int style; const UkPalette
*pal; };
const UkPalette &uk_theme_palette (int theme); // a named theme's colours (its own,
else its style's)
extern const UkNamedTheme uk_themes[]; // ..., { 0, 0, 0 }
static const unsigned UK_GREY = 0x00ACACB0; // the inactive frames

void uk_theme_load (); // (UIKit::init () calls it; idempotent)
void uk_theme_reload (); // read SD:/etc/theme.txt again (a new theme applied:
the dock)
void uk_theme_face (unsigned face); // the palette's shades from a face colour (a
preview)
```

The whole theme as values (the Theme applet edits one, previews it, writes it). UK_AUTO: a colour derived from the window's (button, field, menubar).

```
static const unsigned UK_AUTO = 0xFF000000u;
struct UkTheme
{
    int theme; // uk_themes[] index of the active frame's colour, -1:
    'active'
    unsigned active, inactive, window, button, field, accent, menubar, dock;
    int outline; // 0 none, 1 dark, 2 black
    int style; // the frames' look: UK_STYLE_CDE / UK_STYLE_MILK
};
```

A theme's style changed (a scheme chosen): the colours that are the style's own -- the window, the accent, the frames behind, the dock -- become the new style's (what the user set otherwise is kept); the buttons, the fields and the menu bar follow the window's.

```
void uk_theme_take_style (UkTheme &t, int style);
```

A named theme chosen (uk_themes[theme]): its frame, its style, and -- when its colours are not those of the theme left -- its window, accent, frames behind, dock, buttons, fields, outline.

```
void uk_theme_take (UkTheme &t, int theme);
void uk_theme_defaults (UkTheme &t);
void uk_theme_parse (const char *text, UkTheme &t); // theme.txt's keys over t
void uk_theme_get (UkTheme &t); // the palette in use
void uk_theme_set (const UkTheme &t); // ... made this one (every shade
computed)
```

```
int uk_theme_write (const UkTheme &t, char *out, int cap); // theme.txt's text ->
its length
```

uikit/text.h

uikit/text.h -- the text-rendering hook: a TextFace an app installs (uk_set_textface) so that every text path of uikit draws and measures with it -- Canvas::text, uk_text_l / _c / _w, the line height widgets lay out with (uk_fh ()), the widths they compute (a NumericUpDown's right alignment, tooltips, icon labels, the grid's cells, the text boxes' carets...). Without a face (the default) uikit draws with its bitmap fonts exactly as before, byte for byte.

A face is proportional and its text is UTF-8 (the bitmap path stays Latin-1, one byte a glyph). Styles: 0 regular, 1 italic, 2 bold, 3 bold italic (as the bitmap families). Install it before building the widgets (some size themselves from uk_fh () when made):

```
#include "fontkit/uikitface.h"          (a newlib app linking fontkit/libft.a:
FreeType's anti-aliased text)
int main () { ft_uikit_install ("DejaVu Sans", 13); Root root (...); ... }
```

The window's frame (its title) has a face of its own, the same in every app: SD:/res/fonts/title.aaf (DejaVu Sans Bold rendered by FreeType ahead of time: uikit/skin.cpp), else the desktop's bitmap font.

```
struct TextFace
{
    virtual int height () = 0;          // a line's height, px (uk_fh () while installed)
    virtual int ascent () = 0;          // from a line's top to its baseline, px
    virtual int width (const char *utf8, int style) = 0;          // the advance,
px
    // Draw at x, the line's top at yTop (a line of height () px), blended over the
    canvas.
    virtual void draw (Canvas &cv, int x, int yTop, const char *utf8, unsigned color,
int style) = 0;
    // The width of the first n bytes (a prefix: the caret's place). The default
    copies them and
    // asks width (); a face overrides it to measure in place.
    virtual int widthN (const char *utf8, int n, int style);
    virtual ~TextFace () {}
};
```

```
UIKIT_VAR (TextFace *, uk_face_);      // (the face in use: uk_textface ())
UIKIT_VAR (int, uk_face_fw_);          // (its digit's width: uk_fw () while
installed)
void uk_set_textface (TextFace *f);    // install a face; 0: back to the bitmap
fonts
```

The face uk_set_textface installed, 0 while the bitmap fonts are in use.

```
TextFace *uk_textface ();
```

Another face for the time of a scope -- a widget's own captions, a display's large digits --, the one before back at its end. f = 0: no change.

```
struct UkFaceScope
{
    TextFace *keep; int keepFw;
    UkFaceScope (TextFace *f) : keep (uk_face_), keepFw (uk_face_fw_)
    { if (f) { uk_face_ = f; uk_face_fw_ = f->width ("0", 0); if (uk_face_fw_ < 1)
uk_face_fw_ = 1; } }
};
```

```
~UkFaceScope () { uk_face_ = keep; uk_face_fw_ = keepFw; }
};
```

measuring and drawing, through the face when one is installed

The bitmap fonts' cell (the kernel's font: what uk_fw () / uk_fh () give without a face).

```
int uk_bfw ();
int uk_bfh ();
```

The width of s as Canvas::text draws it (no face: its length x uk_fw ()); of its first n bytes.

```
int uk_tw (const char *s, int style = 0);
int uk_tw_n (const char *s, int n, int style = 0);
```

The place (a byte index <= n, a character's start) whose caret is the nearest to x px from the text's start: a click in a text.

```
int uk_tpos (const char *s, int n, int x, int style = 0);
```

s at (x, y) -- the line's top-left -- in style (the bitmap path: the loaded family's style).

```
void uk_text (Canvas &cv, int x, int y, const char *s, unsigned c, int style = 0);
```

... clipped to the box (cx, cy, cw, ch): a text that runs past a field's edge.

```
void uk_text_clip (Canvas &cv, int x, int y, const char *s, unsigned c, int style,
int cx, int cy, int cw, int ch);
```

Fit s into w px (a copy in out, cap bytes): cut at a character, "..." at its end when it did not fit. Returns its width.

```
int uk_text_fit (const char *s, int w, char *out, int cap, int style = 0);
```

Word-wrap the first n bytes of s into lines of w px at most, maxLines at most: line k is the bytes [start[k], start[k] + len[k]) of s (start and len: maxLines entries each). A '\n' ends a line (an empty line for "\n\n"); a line is broken at its last space that fits (the spaces at the break dropped); a word wider than w is cut at a character (UTF-8). *more (when given): text is left past the last line (more than blanks). → the number of lines. (A card showing a paragraph: Stickies.)

```
int uk_text_wrap (const char *s, int n, int w, int maxLines, int *start, int *len,
bool *more = 0, int style = 0);
```

s at (x, y) -- the line's top-left -- straight over a see-through canvas (a WIN_FLAG_ALPHA window: the wallpaper shows through; uk_paint_alpha on), its glyphs blended by their coverage with the bitmap fonts or the installed face alike (a face's own draw ignores the alpha mode). shade: 0 none; 1 a soft shadow (for a light ink on a dark wallpaper); 2 engraved: a light edge under each glyph, of back (the wallpaper's colour there; for a dark ink on a light wallpaper). The agenda's header, Stickies'.

```
void uk_text_over (Canvas &cv, int x, int y, const char *s, unsigned ink, int style =
0, int shade = 0, unsigned back = 0);
```

UTF-8 (the face's text)

A sequence's length from its first byte and what follows it (1 for a byte that does not start a valid sequence: read as Latin-1, so older text still shows), bounded by n bytes.

```
int uk_u8_len (const char *s, int n);
```

The character at s (its length in *len), a stray byte as its Latin-1 character.

```
unsigned uk_u8_get (const char *s, int n, int *len);
```

The next / the previous character's start from byte i (of n).

```
int uk_u8_next (const char *s, int i, int n);  
int uk_u8_prev (const char *s, int i);
```

cp written as UTF-8 into o (4 bytes room) → its length.

```
int uk_u8_put (char *o, unsigned cp);
```

A typed key as the text widgets store it with a face: its UTF-8 (Latin-1 0xA0..0xFF, 0x80 the euro sign as the keymaps send it) → the length (0: not a character).

```
int uk_u8_key (long k, char *o);
```

uikit/label.h

uikit/label.h -- non-interactive text on a solid background.

```
class Label : public Widget  
{  
public:  
    char    text[128];  
    unsigned bg, fg;  
    Label (int l, int t, int w, int h, const char *s, unsigned fg_ = C_TEXT, unsigned  
bg_ = C_BG);  
    void setText (const char *s);  
    void onDraw () override;  
};
```

uikit/button.h

uikit/button.h -- a push button (the theme's raised face, as a drop-down's: uikit/paint.h); fires cb on release-over.

```
class Button : public Widget  
{  
public:  
    char    text[64];  
    Action  cb;  
    Button (int l, int t, int w, int h, const char *s, Action cb_ = 0);  
    void onDraw () override;  
    bool onMouse (int mx, int my, int bl, int br, int bm, int wheel) override;  
    bool onKey (long k) override;      // focused (Tab): Space / Enter press it  
};
```

uikit/dialog.h

uikit/dialog.h -- modal dialogs as uikit objects. A dialog is just a normal uikit Widget added as the TOP child of the Root and flagged `modal`, so Root routes all input to it (see Widget::handleMouse/handleKey) until it closes and removes itself. It is built from ordinary uikit controls (Button / Textbox / Scrollbar) for a uniform look.

```
uikit::Modal      -- base: box widget + the nested run() loop.  
uikit::MessageBox -- title + message + OK / Yes-No / OK-Cancel.  
uikit::FileDialog -- navigable file browser (open / save).
```

Convenience: uk_messagebox / uk_file_open / uk_file_save.

```

class Modal : public Widget
{
public:
    bool done; int result;
    Modal (int w, int h);           // box-sized widget, centred + flagged modal by
run()
    int run ();                     // add to Root, pump until closed, remove; returns
result
    void close (int r) { result = r; done = true; }
    virtual void onButton (int tag) { (void) tag; } // from a dialog button
    virtual void onScroll (int value) { (void) value; } // from a dialog scrollbar
    unsigned bgColor () override; // the face (its controls blend into it)
    // The box: a rounded panel of the face, a title strip in the active frame's
colour, an
    // outline; its corners see-through. The title strip's height: titleH ().
    void drawBox (const char *title);
    static int titleH ();
};

class MessageBox : public Modal
{
    const char *m_title, *m_text; int m_def;
public:
    MessageBox (const char *title, const char *text, int buttons);
    void onButton (int tag) override { close (tag); }
    bool onKey (long k) override; // Enter = default, Esc = cancel
    void onDraw () override;
};

```

The file dialog (uk_file_open / uk_file_save / uk_folder_open show it): an Up button and the folder's path, the volumes at the left, the folder's content (the folders first, sorted, the files' sizes); a click selects, a double click / Enter / the button opens a folder or takes the file; the arrows, Backspace (up). startDir may be a file's path: its folder (a save dialog proposes its name). ITS SIZE AND ITS FUNCTIONS ARE THE LIBRARY'S ABI: the fields below are kept as they always were (dialog.cpp says how the dialog uses them), nothing is added.

```

class FileDialog : public Modal
{
    enum { MAXENT = 128 };
    char m_dir[256], m_ent[MAXENT][96], m_isdir[MAXENT];
    int m_count, m_sel, m_top, m_rows, m_rowH;
    int m_lastRow; unsigned m_lastTick; // a double click on a file: confirmed
    int m_lx, m_ly, m_lw, m_lh; // file-list rect (box-local)
    bool m_save, m_folder; // m_folder: pick a directory (no filename box)
    Textbox *m_nameBox;
    Scrollbar *m_sb;
    void read (); void goUp (); void enter (const char *name); void click (int row);
void syncSb ();
public:
    FileDialog (const char *startDir, const char *defName, bool save, bool folder =
false);
    void onScroll (int v) override { m_top = v; invalidate (true); }
    void onButton (int tag) override;
    bool onMouse (int mx, int my, int bl, int br, int bm, int wheel) override; //
list area
    bool onKey (long k) override; // Enter = Open / Save, Esc = Cancel
    void onDraw () override;
    void getResult (char *out, unsigned cap); // dir + "/" + filename (folder
mode: dir)
    // The kinds of files shown: "Text files|.txt;*.md|All files|*" -- pairs of a

```

```

name and its patterns
// (';' between them; "*" : every file), '|' between everything. A drop-down
beside the buttons chooses
// one; the folder shows the files that match it (and every folder). A save: a
name typed without an
// extension takes the kind's first one. 0 / "" : every file, no drop-down. The
text must live as long
// as the dialog (it is not copied).
void setFilters (const char *filters);
const char *fileName () const { return m_nameBox ? m_nameBox->text : ""; } //
the name box's text
};

```

Colour dialog (WPF-style ColorPicker dialog): R / G / B sliders, a 16-colour palette, a preview (old | new) and the hex value; OK / Enter keeps it, Cancel / Esc does not.

```

class ColorDialog : public Modal
{
public:
    unsigned color, orig;
    ColorDialog (unsigned initial, const char *title);
    void onButton (int tag) override { close (tag); }
    bool onKey (long k) override;
    void onDraw () override;
    bool onMouse (int mx, int my, int bl, int br, int bm, int wheel) override;
    void syncSliders ();
    class Slider *r, *g, *b;
private:
    const char *m_title;
};

```

Convenience: spin a one-shot dialog and return its outcome. A pop-up menu at (x, y) of the window (a context menu, the window menu): a floating panel of items -- the one under the pointer in the accent -- and separators. run () shows it until an item is picked (its id), or a click elsewhere / Esc (-1).

```

class PopupMenu : public Modal
{
    enum { MAXI = 16 };
    const char *m_label[MAXI], *m_hint[MAXI]; int m_id[MAXI]; bool m_on[MAXI],
m_sep[MAXI];
    int m_n, m_hot;
    int rowY (int i) const;
    int rowAt (int mx, int my) const;
public:
    PopupMenu (int x, int y);
    void add (const char *label, int id, bool enabled = true, const char *hint = 0);
    void separator ();
    int run (); // fits the menu in the window, shows it -> id / -1
    void onDraw () override;
    bool onMouse (int mx, int my, int bl, int br, int bm, int wheel) override;
    bool onKey (long k) override;
};

```

```

int uk_messagebox (const char *title, const char *text, int buttons); // modal;
buttons: MB_* -> 1 OK / Yes, 0 Cancel / No / Esc (2: Yes-No-Cancel's No)

```

The file dialogs: modal, from startDir (a folder, or a file's path: its folder); true = OK (out: the chosen path). filters: the kinds of files offered, "Text files|.txt;.md|All files|*" (FileDialog::setFilters); 0: all.

```

bool uk_file_open (char *out, unsigned cap, const char *startDir, const char *filters
= 0);
bool uk_file_save (char *out, unsigned cap, const char *startDir, const char
*defName, const char *filters = 0);    // with a name box (defName in it)
bool uk_folder_open (char *out, unsigned cap, const char *startDir);    // pick a
directory
bool uk_color_dialog (unsigned *color, const char *title = "Colour");    // true = OK
(*color set)

```

uikit/bmp.h

bmp.h -- an icon's picture for UIKit and the apps (the docks, the lists, the skins).

ui::icon_load reads a picture file through ImageKit -- any format it reads: the icons are BMP today, they may be PNG tomorrow -- and gives it as the icons are drawn: 0x00RRGGBB, top-down, the see-through pixels in the icons' key colour (magenta, 0x00FF00FF). (Until 2026-10-05 this was user/bmp.hpp, a 24-bit BMP reader compiled into each program.)

The picture of a file (new[]: the caller's to delete[]), its size in *pw, *ph; 0 if it cannot be read.

```

unsigned *icon_load (const char *path, int *pw, int *ph);

```

(the name the programs knew: the same)

```

unsigned *bmp_decode (const char *path, int *pw, int *ph);

```

uikit/calendar.h

uikit/calendar.h -- Calendar: a month grid to pick a date (WPF Calendar); DatePicker: a date field that opens a Calendar below it (WPF DatePicker).

Calendar: < / > (or Page Up / Page Down) change the month; click a day (or arrows + Enter) picks it: year/month/day + cb. Today is outlined.
 DatePicker: shows YYYY-MM-DD; click to open the calendar, pick a day to close it (Esc or a click outside cancels); cb fires on a new date.

```

class Calendar : public Widget
{
public:
    int year, month, day;    // the selected date (month 1..12)
    int viewYear, viewMonth;    // the month shown
    Action cb;
    Calendar (int l, int t, int y, int m, int d, Action cb_);    // size: CAL_W x
    CAL_H
    void setDate (int y, int m, int d);
    void onDraw () override;
    bool onMouse (int mx, int my, int bl, int br, int bm, int wheel) override;
    bool onKey (long k) override;
    static int daysIn (int y, int m);
    static int dayOfWeek (int y, int m, int d);    // 0 = Monday
};
enum { CAL_CELL_W = 28, CAL_CELL_H = 20, CAL_HDR = 24, CAL_W = 7 * CAL_CELL_W + 2,
        CAL_H = CAL_HDR + 18 + 6 * CAL_CELL_H + 4 };

class DatePicker : public Widget
{
public:
    int year, month, day; Action cb; bool open;
    DatePicker (int l, int t, int w, int h, int y, int m, int d, Action cb_);
    ~DatePicker ();
    void onDraw () override;

```

```

    bool onMouse (int mx, int my, int bl, int br, int bm, int wheel) override;
    bool onKey (long k) override;
    void setOpen (bool o);
    void format (char *out) const;          // "YYYY-MM-DD"
    Calendar *cal;                        // the drop-down: made at the first opening, a child
while open
    int rowH, origW;
};

```

uikit/checkbox.h

uikit/checkbox.h -- a box + check mark + label; toggles on click.

```

class Checkbox : public Widget
{
public:
    char    text[64]; bool checked; Action cb; unsigned bg;
    Checkbox (int l, int t, int w, int h, const char *s, bool chk, Action cb_,
unsigned bg_ = C_BG);
    void onDraw () override;
    bool onMouse (int mx, int my, int bl, int br, int bm, int wheel) override;
    bool onKey (long k) override;          // Space / Enter toggle
};

```

uikit/codeedit.h

uikit/codeedit.h -- CodeEdit: a code editor. BASIC (or QBStudio's .form text) in a monospaced face, the language's colours, the line numbers, the indentation's guides, the lines with a problem marked (a red dot in the margin, the text underlined), a line lit (an error's, a debugger's current line), undo / redo, the clipboard, and the completion (after a name and a dot, or Ctrl+Space: the program gives the items). The words of the language are written in capitals as you type (as QBasic does) when isKeyword is set. Made from QBStudio's editor (2026-10-06) for every program that edits code: QBStudio, Turtle Quest.

```

CodeEdit *ed = new CodeEdit (x, y, w, h);
ed->mono = myMonoFace; ed->isKeyword = is_basic_word; // (0: the UI's face; no
keyword colours)
ed->setText (src); ed->onChange = changed;
ed->hiLine = 4; ed->invalidate (true);                // line 4 lit (1-based; 0
none)
ed->clearMarks (); ed->addMark (7);                    // line 7 has a problem

```

Keys: the arrows (Ctrl: by word), Home / End (Ctrl: the text's), Page Up / Down, Shift: select; Ctrl+A, C, X, V, Z, Y; Tab / Shift+Tab: indent / unindent the lines chosen; Enter keeps the indentation (two spaces more after a line that opens a block); Ctrl+Space: the completion. Mouse: click, drag, double click (a word), the wheel, the scroll bar.

MIT License -- Copyright (c) 2026 Stéphane Wegener and the Onyx contributors (docs/LICENSING.md).

```

enum { CODE_BASIC = 0, CODE_FORM = 1 };          // CodeEdit::lang (CODE_FORM:
QBStudio's .form text)
struct CodeEditState;                            // (the editor's text, lines, undo:
codeedit.cpp)

class CodeEdit : public Widget
{
public:
    bool    readonly;
    int    lang;                                // CODE_BASIC / CODE_FORM

```

```

int hiLine; // a line lit (1-based; 0 none)
Action onChange; // the text changed
Action onCaret; // the caret moved
TextFace *mono; // the code's face (0: the UI's)
TextFace *ui; // the completion list's face (0: the UI's)
bool (*isKeyword) (const char *w, int n); // a word of the language?
(colours, capitals; 0: none)
bool (*isFlag) (const char *w, int n); // CODE_FORM: a flag word (0: none)
// The completion: called with the name before the dot ("" for Ctrl+Space); it
calls addCompletion for each
// item -- kind: 'p' property, 'm' method, 'c' control, 's' a SUB, 'k' a word,
'f' a function, 't' a type...
void (*complete) (CodeEdit &ed, const char *object);
void *user; // the program's (e.g. the document this editor
shows)

```

```

CodeEdit (int l, int t, int w, int h);
~CodeEdit () override;

```

```

const char *text () const;
int length () const;
void setText (const char *s); // a new text: no undo, the caret at the
start
void replaceAll (const char *s, int caret); // the whole text replaced as one
step that can be undone
void insertText (const char *s); // at the caret, replacing the selection
int caret () const;
int caretLine () const; // 0-based
int caretCol () const;
int lineCount () const;
int lineStart (int ln) const; // the offset of a line (0-based)
void gotoLine (int ln, bool select = false); // the caret at a line's text (0-
based), shown in the middle
void setCaret (int p);
void showLine (int ln); // a line (0-based) scrolled into view, the
caret left where it is
bool hasSel () const;
void selectAll ();
void copy ();
void cut ();
void paste ();
void undo ();
void redo ();
bool find (const char *w); // the next w (any case) from the caret,
chosen; false: none
void wordAt (int p, char *obj, int ocap, int *wordStart); // the name before p,
and the object before its dot
// The lines with a problem (1-based)
void clearMarks ();
void addMark (int line);
bool hasMark (int line) const;
void addCompletion (const char *name, char kind, const char *detail); // (from
complete)

```

```

void onDraw () override;
bool onMouse (int mx, int my, int bl, int br, int bm, int wheel) override;
bool onKey (long k) override;
void resizeTo (int w, int h) override;
private:
friend struct CodeEditOps;

```

```

CodeEditState *m;
unsigned long m_reserved[4] = { 0, 0, 0, 0 }; // (uikit/abi.h)
};

```

uikit/colorpick.h

uikit/colorpick.h -- a colour swatch; clicking expands a fixed 8×5 palette grid below it (the widget grows + comes to front while open, grabs outside clicks to close). cb fires when a colour is picked; read it from `color`.

```

class ColorPicker : public Widget
{
public:
    unsigned color; bool open; Action cb;
    ColorPicker (int l, int t, int w, int h, unsigned initial, Action cb_);
    void onDraw () override;
    bool onMouse (int mx, int my, int bl, int br, int bm, int wheel) override;
private:
    int m_boxW, m_boxH;
    void setOpen (bool o);
};

```

uikit/combobox.h

uikit/combobox.h -- an editable Textbox with a drop-down list of suggestions (WinForms ComboBox, DropDown style): type any text, or click the arrow (or press Down / Up) to pick one of the options → the text becomes that option and onPick fires with `picked` = its index. While open the widget grows downward over its siblings (like Dropdown).

```

class Combobox : public Textbox
{
public:
    enum { MAXOPT = 16, ARROW_W = 20 };
    char      opts[MAXOPT][64]; int nopts, picked, rowH; bool open; Action onPick;
    Combobox (int l, int t, int w, int h, const char *s = "", Action enter = 0,
Action pick = 0);
    void clearOptions ();
    void addOption (const char *s);
    void pick (int i); // set the text to option i, fire onPick
    void onDraw () override;
    bool onMouse (int mx, int my, int bl, int br, int bm, int wheel) override;
    bool onKey (long k) override;
private:
    int m_hot; // the list's row under the pointer (-1 none)
    bool m_arrowHot = false; // the pointer over the arrow
    int rowAt (int mx, int my) const;
    void setOpen (bool o);
};

```

uikit/datagrid.h

uikit/datagrid.h -- DataGrid: a read-only table of rows and columns (WPF's DataGrid, a file manager's details view, a database's list). Virtual: the app says how many rows there are and gives each cell's text (`cellText`) -- or draws the cell itself (`cellDraw`: a colour swatch, a check mark...). On top, the header: the columns' titles in bold; a click on a title asks the app to sort (`onSort`, `clickedCol`: the app orders its rows, the grid shows `sortCol` / `sortDesc` as an arrow); a drag on a title's right edge resizes its column. Below, the rows: every other one a shade darker (stripes), a light line between the columns, the one under the pointer tinted, the selected one in the accent (dimmer while the grid has not the focus); both scroll bars when the rows or the columns do not fit (their thumbs drag, a click on the groove jumps there).

Mouse: a click selects (onSelect), a double click activates (onActivate), a right click selects the row under it and asks for a menu (onContext: ctxRow -- -1 below the rows --, ctxX, ctxY: where, in the grid's coordinates); the wheel scrolls (with Shift: sideways).
 Keys (focused): Up / Down / Page Up / Page Down / Home / End move the selection (onSelect), Left / Right scroll sideways, Enter activates; the others go to the parent (Delete...).

A cell's text is cut to its column with "..." (the app's text: control characters shown as spaces). Integer only, as the rest of uikit.

```
DataGrid *g = new DataGrid (0, 0, 600, 400);
g->setColumns (2);
g->setColumn (0, "Name", 200); g->setColumn (1, "Size", 90, GRID_RIGHT);
g->cellText = my_text; // const char *my_text (DataGrid &g, int row, int
col, char *buf, int cap)
g->setRows (n);
```

```
enum { GRID_LEFT = 0, GRID_RIGHT = 1, GRID_CENTRE = 2 };
```

```
class DataGrid : public Widget
{
public:
    enum { MAXCOLS = 64 };
    struct Column { char title[48]; int width, align; };
    // A cell's text: written into buf (cap bytes) and returned, or a constant string
    returned.
    typedef const char *(*CellText) (DataGrid &g, int row, int col, char *buf, int
    cap);
    // A cell drawn by the app: cv is clipped to the cell, whose box is (x, y, w, h)
    there; ink is
    // the text's colour (on the selection or not). Return false to have the text
    drawn instead.
    typedef bool (*CellDraw) (DataGrid &g, Canvas &cv, int row, int col, int x, int
    y, int w, int h,
        unsigned ink, bool selected);
```

```
    int nrows, sel, top, left; // the rows; the selected one (-1 none); the
    first shown; px scrolled sideways
    int rowH, headH; // (default: the font's height + 8, + 10)
    int sortCol; bool sortDesc; // the title showing the sort's arrow (-1
    none)
    bool sortable, stripes; // titles clickable (onSort); every other row
    shaded
    CellText cellText; CellDraw cellDraw;
    Action onSelect, onActivate, onSort, onContext;
    int clickedCol; // (onSort) the title clicked
    int ctxRow, ctxX, ctxY; // (onContext) the row right-clicked (-1 none),
    where
    const char *emptyText; // written in the body when there is no row (0:
    nothing)
    void *user; // the app's (e.g. which table the grid shows)
```

```
DataGrid (int l, int t, int w, int h);
void setColumns (int n); // n columns (untitled, 100 px wide, left-
aligned)
int columns () const { return m_ncol; }
```

```

    void setColumn (int c, const char *title, int width, int align = GRID_LEFT);
    Column &column (int c) { return m_col[c < 0 ? 0 : c >= MAXCOLS ? MAXCOLS - 1 :
c]; }
    // Column c as wide as its title and the texts of its first `sample` rows need,
    within [minW, maxW].
    void autoSize (int c, int minW, int maxW, int sample = 300);
    void setRows (int n);           // the number of rows (the selection kept among
    them)
    void setSel (int r);           // select row r (-1: none) and scroll it into
    view; no callback
    void ensureVisible (int r);
    int rowAt (int y) const;       // the row under y (grid coordinates), -1 none
    int totalWidth () const;       // the columns' widths added
    int visibleRows () const;      // how many rows the body shows
    void onDraw () override;
    bool onMouse (int mx, int my, int bl, int br, int bm, int wheel) override;
    bool onKey (long k) override;
private:
    Column m_col[MAXCOLS]; int m_ncol;
    int m_hotRow, m_hotHead;       // under the pointer (-1 none)
    int m_drag;                   // what the left button holds: a G_* below
    int m_dragCol, m_dragX0, m_dragW0;
    unsigned m_lastClick; int m_lastRow;
    bool m_rdown;
    // The geometry: the body (the rows, below the header) and the scroll bars.
    void bars (bool *vbar, bool *hbar, int *bodyW, int *bodyH) const;
    int colX (int c) const;        // a column's left edge in the body (scrolled)
    int headAt (int mx) const;     // the title under x, -1
    int edgeAt (int mx) const;     // the column whose right edge is under x, -1
    void clampScroll ();
    void pick (int r, bool fire);
};

```

uikit/dropdown.h

uikit/dropdown.h -- a non-editable drop-down list (the sibling of Combobox, same look): a closed box showing the selected option + a drop button; clicking expands a list below it (the widget grows downward while open and grabs outside clicks to close) -- above it when it does not fit below in the window and fits better there. Open, the list is over everything in the window (the widget is the window's child meanwhile -- the dialog's when it is in one --: uikit/lift.h): no parent clips it, nothing is drawn over it, wherever the box stands. Keyboard (when focused): Up / Down pick the previous / next option, Enter or Space opens / closes the list, Esc closes it. cb fires when the selection changes. The option strings are NOT copied: they must outlive the widget.

```

class Dropdown : public Widget
{
public:
    const char *const *opts; int nopts, sel, rowH; bool open; Action cb;
    Dropdown (int l, int t, int w, int h, const char *const *options, int n, int
    initial, Action cb_);
    void setOptions (const char *const *options, int n, int initial);
    void onDraw () override;
    bool onMouse (int mx, int my, int bl, int br, int bm, int wheel) override;
    bool onKey (long k) override;
private:
    int m_hot;                   // the row under the pointer while open (-1 none)
    bool m_up;                   // open upward (the box at the bottom, `top` moved up)
    int m_top0;                  // ... its `top` when closed
    int listTop () const;        // the open list's first row
    int boxY () const { return m_up ? height - rowH : 0; } // the box's row

```

```

    int rowAt (int mx, int my) const; // the row at (mx, my) while open, or -1
    void setOpen (bool o);
};

```

The open list of a drop-down (Dropdown, Combobox): a floating panel at y0, n rows of rowH (sel: the chosen one, hot: the one under the pointer, -1 none).

```

void uk_draw_option_list (Canvas &cv, int y0, int w, int rowH, const char *const
*opts, int n,
                        int sel, int hot);

```

uikit/flat.h

uikit/flat.h -- UIKit for a program that is not C++: a window and its widgets as handles, plain C functions on them. It is what Onyx BASIC calls (#import uikit: UIKit.window, UIKit.button ...; the code QBStudio makes of a form), and what a C program may call.

```

void *win = uk_window ("Hello", 320, 120, 0);
uk_label (win, 12, 12, 200, 20, "Your name:");
void *name = uk_textbox (win, 12, 36, 200, 24, "", 0);
uk_button (win, 220, 36, 80, 24, "OK", on_ok); // void on_ok (void
*button)
uk_window_run (win); // until the window is
closed

```

A handle is a pointer the program keeps and gives back; 0 is "none" (a function given 0 does nothing). A program has one window. The widgets belong to their window and end with it. A text is Latin-1. A callback is called from uk_window_run (or uk_window_step), on the program's own thread.

MIT License -- Copyright (c) 2026 Stéphane Wegener and the Onyx contributors (docs/LICENSING.md).

```

typedef void (*uk_event) (void *widget); // a widget's event: the widget it
comes from
typedef void (*uk_resized) (void *window, int w, int h);
typedef void (*uk_chosen) (void); // a menu item chosen

```

the window

```

enum { UK_FLAT_RESIZABLE = 1 };
void *uk_window (const char *title, int w, int h, int flags); // the program's
window (w x h: its inside); 0: none
void uk_window_run (void *window); // the events, until the window is closed
(its box, uk_window_close)
int uk_window_step (void *window); // ... one round of them, for a program
with its own loop: 0 once closed
int uk_window_wait (void *window); // ... one round and the wait before the
next (16 ms): WHILE uk_window_wait (w) ... WEND
void uk_window_close (void *window); // uk_window_run returns,
uk_window_step gives 0
int uk_window_width (void *window);
int uk_window_height (void *window);
void uk_window_min_size (void *window, int w, int h); // the smallest a resizable
window goes
void uk_window_on_resize (void *window, uk_resized fn); // after the user resized it:
place the widgets again

```

the widgets (x, y, w, h: their place in what holds them)

What holds a widget -- the `window` every maker takes -- is the window itself or a panel: a plain area with widgets of its own, placed from its top left corner. Hiding, showing, moving a panel does it to all it holds: a page of an app is a panel, shown in place of another.

```
void *uk_panel (void *window, int x, int y, int w, int h);
void uk_set_parent (void *widget, void *window); // the widget taken from what
holds it and put into a window or a panel
int uk_width (void *widget); // a widget's, a panel's, the window's
size
int uk_height (void *widget);
void *uk_label (void *window, int x, int y, int w, int h, const char *text);
void *uk_button (void *window, int x, int y, int w, int h, const char *text, uk_event
on_click);
void *uk_textbox (void *window, int x, int y, int w, int h, const char *text,
uk_event on_change);
void *uk_checkbox (void *window, int x, int y, int w, int h, const char *text, int
checked, uk_event on_click);
void *uk_listbox (void *window, int x, int y, int w, int h, const char *items,
uk_event on_change); // items: "one|two|three"
void *uk_dropdown (void *window, int x, int y, int w, int h, const char *items,
uk_event on_change);
void *uk_slider (void *window, int x, int y, int w, int h, int max, int value,
uk_event on_change); // from 0 to max
void *uk_progress (void *window, int x, int y, int w, int h, int max, int value);
void uk_set_range (void *widget, int min, int max); // a slider, a progress bar: its
ends
```

A widget's text: a label's, a button's, a checkbox's, what a text box holds, the chosen item of a list or a drop-down. `uk_get_text`'s result is good until the next call of `UIKit`.

```
void uk_set_text (void *widget, const char *text);
const char *uk_get_text (void *widget);
```

Its number: a checkbox's state (1 / 0), the chosen item's place in a list or a drop-down (from 0; -1: none), a slider's or a progress bar's value.

```
void uk_set_value (void *widget, int value);
int uk_get_value (void *widget);
void uk_add_item (void *widget, const char *text); // a list: one more item
void uk_clear_items (void *widget); // a list: emptied
int uk_item_count (void *widget); // a list, a drop-down
void uk_move (void *widget, int x, int y, int w, int h);
void uk_show (void *widget, int on);
void uk_enable (void *widget, int on);
int uk_shown (void *widget);
int uk_enabled (void *widget);
void uk_focus (void *widget);
```

the menus (in the screen's menu bar while the window is the active one)

One more item in the menu `title` (made at its first item); item "-": a separator; key: "Ctrl+S" or "".

```
void uk_menu_item (void *window, const char *title, const char *item, const char
*key, uk_chosen on_choose);
```

the dialogs

```
int uk_message (const char *title, const char *text, int buttons); // buttons 0 OK,
1 OK Cancel, 2 Yes No, 3 Yes No Cancel -> 1 OK / Yes, 0 Cancel / No (3: 2 for No)
```

```

const char *uk_ask_open (const char *folder);           // the file chosen ("": none);
good until the next call
const char *uk_ask_save (const char *folder, const char *name);

}
#endif

```

uikit/font.h

uikit/font.h -- Font: a bitmap font *family* loaded from one Onyx ".fnt" file on the SD card (e.g. SD:/fonts/ns-sans.fnt, the MIT "ns-sans" face exported by tools/fonts/ gen_nssans.py from NetSurf's glyph_data). One file embeds all four styles; the header carries the glyph size and the file offset of each style block:

```

0  "ONYF"  magic
4  version (u8) | width (u8) | height (u8) | styles (u8)
8  glyphs  (u16 LE, per style) | reserved (u16)
12 offset[styles] (u32 LE each): file offset of each style block
    style index = (bold?2:0)|(italic?1:0): 0=reg 1=italic 2=bold 3=bold+italic
..  style blocks: glyphs * height bytes, 1 byte/row, bit 0x80 = leftmost px.

```

(A legacy 2-byte-header single-style .fon is also accepted, as one regular style.)

Draw with Canvas::drawFont (declared in canvas.h, implemented in font.cpp). A global default family is loaded once by uikit::init() and reachable via uikit::font(), so any widget can render styled text from a real face, not just the kernel's single font.

```

class Font
{
public:
    static const int MAXSTYLE = 4;
    unsigned char *data;           // whole file image; 0 = not loaded
    int gw, gh, gcount, nstyles;   // glyph width/height, glyphs per style, style
count
    unsigned off[MAXSTYLE];        // byte offset (within data) of each style block

    Font ();
    ~Font ();

    bool load (const char *path);   // (re)load an Onyx .fnt/.fon
    bool valid () const { return data != 0 && nstyles > 0; }
    int width () const { return gw; }
    int height () const { return gh; }
    int styles () const { return nstyles; }
    // gh bytes (MSB-left rows), or 0. style 0=reg 1=italic 2=bold 3=bold+italic; a
style
    // beyond what this family carries falls back to regular.
    const unsigned char *glyph (unsigned ch, int style = 0) const;

    // Named global families (loaded by uikit::init from SD:/fonts/<name>.fnt).
Convenience
    // over font(id); a family whose file is absent falls back to Sans when drawn.
    static Font &Sans ();
    static Font &Serif ();
    static Font &Cursive ();
    static Font &Mono ();
};

```

The global named font registry. `uikit::init()` loads every known family once (idempotent; Root calls it at startup, so EVERY uikit app gets the fonts -- an app that builds no Root must call `uikit::init()` itself). A widget chooses a face by passing `font(id)` to `drawFont`.

```
enum { FONT_SANS = 0, FONT_SERIF, FONT_CURSIVE, FONT_MONO, FONT_COUNT };
void init (); // load the theme (SD:/etc/theme.txt) and the font families; only the
first call works
Font &font (int id = FONT_SANS); // the family for `id`, or Sans if it isn't
loaded
```

Draw `s` into a raw `0x00RRGGBB` framebuffer (`W x H`) via the font registry -- for app-drawn windows that don't own a uikit Canvas. Scaled by `scale`, in `style`, family `id`. Falls back to the kernel font (`kapi_draw_text_buf`) when the family isn't loaded.

```
void draw_text (unsigned *fb, int W, int H, int x, int y, const char *s, unsigned
color,
               int scale = 1, int style = 0, int id = FONT_SANS);
```

uikit/global.h

`uikit/global.h` -- uikit's global variables (the palette, the installed text face: `uikit/globals.inc`).

They are the PROGRAM's variables: a program is linked at a fixed address and reads them directly (`C_BG`, `uk_face...`), also from the toolkit's inline code compiled into it. When uikit is the shared library `SD:/lib/uikit.so` (`ONYX_LIB_BUILD`; `docs/SHARED-LIBS-PLAN.md`) its code cannot name a program's address: each variable is then a REFERENCE, bound when the library starts in a process to the address the program handed over (`user/Runtime/lib.h` `TLibImports.data`, in `globals.inc`'s order) -- or to a copy of the library's own for a variable added after the program was built.

```
#define UIKIT_VAR(type, name) extern type &name
#define UIKIT_VAR(type, name) extern type name
```

uikit/groupbox.h

`uikit/groupbox.h` -- `GroupBox`: a titled frame around a group of controls (WPF `GroupBox`). Add the controls as its children; their (0,0) is the box's top-left, the frame line runs at `y = font height / 2` and the content starts below the title (`contentTop()`).

```
class GroupBox : public Widget
{
public:
    char title[48]; unsigned bg, frame; // frame: 0 = etched (the default), else a
line's colour
    GroupBox (int l, int t, int w, int h, const char *title_, unsigned bg_ = C_BG);
    int contentTop () const { return uk_fh () + 4; }
    void onDraw () override;
    unsigned bgColor () override { return bg; }
};
```

uikit/icon.h

`uikit/icon.h` -- a magenta-keyed BMP centred above an optional label; hover/press highlight + an optional "running" badge.

```
class Icon : public Widget
{
public:
    char text[64]; unsigned *pix; int iw, ih; bool badged; Action cb; unsigned
bg;
```

```

    Icon (int l, int t, int w, int h, const char *bmp, const char *label, Action cb_,
unsigned bg_ = C_BG);
    ~Icon () override;
    void setBadge (bool b);
    void setIcon (const char *bmp); // (re)load the image (0/"" clears); repaints
    void onDraw () override;
    bool onMouse (int mx, int my, int bl, int br, int bm, int wheel) override;
};

```

uikit/imagebox.h

uikit/imagebox.h -- ImageBox: shows an image (WPF Image). load (path) reads BMP / GIF / PNG / JPEG / PCX / WebP (img/imgload.hpp -- first frame), or setPixels () takes 0x(AA)RRGGBB pixels (copied). stretch: IMG_NONE (1:1, centred, clipped), IMG_FIT (uniform, never enlarged beyond 1:1 unless grow), IMG_FILL (stretched to the box). Transparent pixels show the box background.

```
enum { IMG_NONE, IMG_FIT, IMG_FILL };
```

```

class ImageBox : public Widget
{
public:
    int stretch; bool grow; unsigned bg;
    ImageBox (int l, int t, int w, int h, int stretch_ = IMG_FIT, unsigned bg_ =
C_BG);
    ~ImageBox ();
    bool load (const char *path); // false: unreadable (the box shows nothing)
    void setPixels (const unsigned *px, int w, int h, bool hasAlpha = true);
    void clearImage ();
    int imageWidth () const { return m_w; }
    int imageHeight () const { return m_h; }
    void onDraw () override;
private:
    unsigned *m_px; int m_w, m_h;
};

```

uikit/knob.h

uikit/knob.h -- Knob: a rotary control (a studio's gain, pan, rate...). A 270-degree track (open at the bottom), the value's arc on it in the accent (or arcColor) -- from the start, or from the value 0 for a bipolar one (a pan: the top) --, a round cap with a pointer; an optional label and the value's text under it. Integer values in [vmin, vmax] (the apps are integer-only: a gain of -60.0 .. +6.0 dB is -600 .. 60, shown by a format callback).

```

Knob *k = new Knob (x, y, 48, 64, -600, 60, 0, onGain);
k->setLabel ("Gain"); k->format = fmt_db; k->setDefault (0);

```

Mouse: drag up / down (the whole range in 200 px; Shift held: 1000 px, fine), the wheel (a step), a double click: back to the default (setDefault). Keys when focused: Up / Right, Down / Left (a step), Page Up / Down (ten), Home / End (the ends), Delete (the default). onChange fires at each change of value. Sizes: the dial is the widget's width, less the captions' lines under it (about 24 .. 64 px across).

```

class Knob : public Widget
{
public:
    int value, vmin, vmax;
    int step; // the wheel's / the keys' step (default: a 100th of the
range)
    int def; bool hasDef; // the default (a double click, Delete); setDefault
    bool bipolar; // the arc from 0 (a pan, a detune: the top), not

```

```

from the start
    bool      showValue;          // the value's text under the dial (and the label)
    unsigned  arcColor;           // the value's arc (UK_AUTO: the accent)
    TextFace *face;              // the captions' face (0: uikit's -- the installed
face or the bitmap font)
    void      (*format) (int value, char *out, int cap); // the value's text (0: the
number)
    Action    onChange;

Knob (int l, int t, int w, int h, int lo, int hi, int val, Action cb = 0);
void setValue (int v, bool fire = false); // clamped; repainted; onChange if
fire and it changed
void setRange (int lo, int hi);
void setDefault (int v) { def = v; hasDef = true; }
void setLabel (const char *s);
const char *label () const { return m_label; }
void valueText (char *out, int cap) const; // what is shown under the dial

void onDraw () override;
bool onMouse (int mx, int my, int bl, int br, int bm, int wheel) override;
bool onKey (long k) override;
private:
    char      m_label[24];
    bool      m_drag, m_fine;
    int       m_y0, m_v0;          // the drag's start: its y, the value then
    unsigned  m_lastClick;         // ticks (a double click)
    int lines () const { return (m_label[0] ? 1 : 0) + (showValue ? 1 : 0); }
};

```

uikit/lang.h

uikit/lang.h -- an app's words in the user's language. The sources keep their English words, wrapped: TR ("Save") is the word in the language chosen, else the English one. TRC ("status", "Open") for a word whose translation depends on where it stands (the key "status|Open" in the catalogue).

The catalogues are UTF-8 text files, a line a word: "Englishtranslation" (\t, \n, \ escaped; "#" begins a comment line):

SD:/res/lang/<code>.txt	uikit's own words (the dialogs' buttons, the months...)
SD:/apps/<app>.app/lang/<code>.txt	the app's

The language is the system's (SD:/etc/system.ini's "language=", systemkit/locale.h: the Control Panel's Language & Region applet, Setup); none said: English. An app has no choice of its own. (An app ported to another system -- the Mac's Ledger -- keeps its own there: SD:/apps/.app/lang.txt, uk_lang_choose.) An app drawing with uikit's bitmap fonts gets the words in Latin-1 (as they draw it), one with a text face (FreeType's) in UTF-8: install the face before uk_lang_init. tools/lang/check.py says the words of an app its catalogue has not.

```

const char *uk_tr (const char *en);          // the translation, else en itself
const char *uk_trc (const char *ctx, const char *en); // "ctx|en"'s, else en's,
else en
#define TR(s)      uk_tr (s)
#define TRC(c, s)  uk_trc (c, s)
#define TRN(s)     (s) // a word kept in English in a table and given to TR
() where it is drawn: marked for tools/lang/check.py

void      uk_lang_init ();                    // the system's language loaded -- before

```

```

the UI is built
bool      uk_lang_load (const char *code);          // that language's catalogues
("en": none) -> false: none found
const char *uk_lang ();                          // the language in use: "en", "fr"...
const char *uk_lang_chosen ();                    // the system's (locale_language: "en"
when none is said)
bool      uk_lang_choose (const char *code);       // the system's language changed
(locale_set_language; taken by the programs started next)

```

uikit/layout.h

uikit/layout.h -- automatic layout containers (Panels that re-flow their children on resize and on child add/remove, via the Widget::layout() hook). The children stay passive: they never recompute their own geometry -- the container places them, so a resize cascades down for free (e.g. a calculator filling the secondary section).

```

VerticalStackPanel  -- single column, children top->bottom, full width, each keeps
                    its own HEIGHT (no fixed row height).
HorizontalStackPanel -- single row, children left->right, full height, each keeps
                    its own WIDTH (no fixed column width).
UniformGridLayout   -- cols columns of uniform cells, filled in reading order
                    (left->right, top->bottom). Cell WIDTH derives from the
                    panel width; cell HEIGHT is either a fixed `rowH` (px) or,
                    when `rows > 0`, the panel height divided into `rows` rows.
                    A child may span several columns/rows via its colSpan /
                    rowSpan (default 1) -- e.g. a calculator display spanning 4
                    of 5 columns. Reflows on resize / child add.
                    NB: rowSpan only sizes the child taller; it does not
reserve
                    the cells below for following children (simple flow model).

```

All take a `pad` (outer margin) and `gap` (inter-child spacing), public for tuning.

```

class VerticalStackPanel : public Panel
{
public:
    int pad, gap;
    VerticalStackPanel (int l, int t, int w, int h, unsigned bg = C_BG, int pad_ = 0,
int gap_ = 4)
        : Panel (l, t, w, h, bg), pad (pad_), gap (gap_) {}
    void layout () override;
};

class HorizontalStackPanel : public Panel
{
public:
    int pad, gap;
    HorizontalStackPanel (int l, int t, int w, int h, unsigned bg = C_BG, int pad_ =
0, int gap_ = 4)
        : Panel (l, t, w, h, bg), pad (pad_), gap (gap_) {}
    void layout () override;
};

class UniformGridLayout : public Panel
{
public:
    int cols, rows, rowH, pad, gap;          // rows > 0 -> cell H = height/rows; else
rowH (px)
    UniformGridLayout (int l, int t, int w, int h, int cols_, int rowH_,
unsigned bg = C_BG, int pad_ = 4, int gap_ = 4)

```

```

        : Panel (l, t, w, h, bg), cols (cols_ < 1 ? 1 : cols_), rows (0), rowH (rowH_),
        pad (pad_), gap (gap_) {}
    void setRows (int n) { rows = n; layout (); invalidate (true); } // divide
height into n rows
    void layout () override;
};

```

uikit/lcd.h

uikit/lcd.h -- LcdDisplay: a label drawn as a studio's time / position display: a sunken well (dark in a dark theme, the accent's pale tint in a light one), large digits in the accent (a face given to it, else the bitmap font scaled up), and at its right a small caption over a second line ("BAR.BEAT.16" over "0:14.83").

```

LcdDisplay *pos = new LcdDisplay (x, y, 190, 40, "6.3.2", "BAR.BEAT.16");
pos->setSub ("0:14.83"); ... pos->setText ("6.3.3"); (repainted only when it
changed)
pos->face = bigFace; (e.g. an FtTextFace at 24 px: fontkit/uikitface.h)

```

```

class LcdDisplay : public Widget
{
public:
    TextFace *face; // the digits' face (0: the bitmap font, scaled)
    TextFace *smallFace; // the caption's and the second line's (0:
uikit's)
    int scale; // the bitmap digits' scale (0: from the height, as large
as fits)
    unsigned ink; // the digits' colour (UK_AUTO: the accent, for the
well)
    bool centred; // the digits centred in the room left (else from the
left)

```

```

    LcdDisplay (int l, int t, int w, int h, const char *text = "", const char
*caption = 0);
    void setText (const char *s);
    void setCaption (const char *s);
    void setSub (const char *s);
    const char *text () const { return m_text; }

```

```

    void onDraw () override;
private:
    char m_text[32], m_cap[32], m_sub[32];
};

```

uikit/listbox.h

uikit/listbox.h -- ListBox: a scrollable list of text items with a selection (WPF ListBox). Click selects (onSelect), double-click or Enter activates (onActivate); Up / Down / Page Up / Page Down / Home / End move; the wheel and the scrollbar (drag the thumb or click the track) scroll. Items are copied (64 chars max each).

```

class ListBox : public Widget
{
public:
    int count, sel, top; Action onSelect, onActivate;
    ListBox (int l, int t, int w, int h, Action onSelect_ = 0, Action onActivate_ =
0);
    ~ListBox ();
    void add (const char *s);
    void clear ();

```

```

    const char *item (int i) const { return (i >= 0 && i < count) ? m_items[i] : "";
}

void setSel (int i);           // select + scroll into view (no callback)
void onDraw () override;
bool onMouse (int mx, int my, int bl, int br, int bm, int wheel) override;
bool onKey (long k) override;
private:
    char (*m_items)[64]; int m_cap;
    unsigned m_lastClick; int m_lastIdx; bool m_thumb;
    int rows () const;
    void scrollTo (int t);
    void pick (int i, bool fire);
};

```

uikit/menu.h

uikit/menu.h -- an app's menus for the SYSTEM menu bar (top of the screen, the `menubar` app). Build them once after the Root window exists, then `publish()`:

```

static Menu menu;
menu.menu ("File");
menu.item ("Open...", "^O", UK_CTRL ('O'), onOpen);
menu.separator ();
menu.item ("Save",      "^S", UK_CTRL ('S'), onSave);
menu.publish ();

```

The menu bar shows them while this app is the active one and sends the chosen item back (`GUI_EVENT_MENU`); Menu runs the item's callback. Menu also handles the items' keyboard shortcuts (Root routes every key through `Menu::shortcut` first) and `^Q` = Quit. The bar adds the app-name menu with "Quit" itself.

```

#define UK_CTRL(c) ((long) ((c) & 0x1F)) // Ctrl+letter key code (^A = 1 ... ^Z = 26)

```

```

typedef void (*MenuAction) (); // a menu item's callback (chosen in the bar, or its shortcut typed)

```

```

class Menu
{
public:
    Menu ();
    void menu (const char *title);           // start a new menu
    void item (const char *label, const char *shortcutText, long key, MenuAction cb);
    void separator ();
    void publish ();                         // hand the menus to the menu bar
    bool shortcut (long key);                // run the item bound to `key`
    static Menu *current ();                // the published menu (or 0)
private:
    enum { MAXITEMS = 64 };
    char      m_spec[WIN_MENU_MAX_USER];
    int       m_len;
    MenuAction m_cb[MAXITEMS];
    long      m_key[MAXITEMS];
    int       m_count;
    void put (const char *s);
    static void handler (unsigned long, int ev, gui_value v);
};

```

uikit/numeric.h

uikit/numeric.h -- NumericUpDown: an integer field with up/down arrows (WPF-style spinner). Arrows / the wheel / Up / Down / Page Up / Page Down step it; typing digits (and '-') edits it, Enter or leaving the field commits. Clamped to [vmin, vmax]; cb fires on every change of `value`.

```
class NumericUpDown : public Widget
{
public:
    int value, vmin, vmax, step; Action cb;
    NumericUpDown (int l, int t, int w, int h, int lo, int hi, int val, int step_,
Action cb_);
    void setValue (int v);           // clamp, repaint, fire cb if it changed
    void onDraw () override;
    bool onMouse (int mx, int my, int bl, int br, int bm, int wheel) override;
    bool onKey (long k) override;
private:
    char m_edit[16]; int m_elen;     // digits being typed (m_elen < 0: not
editing)
    int m_down;                     // the arrow held: 1 up, 2 down, 0 none
    void commit ();
};
```

uikit/paint.h

uikit/paint.h -- the procedural painter of the desktop's look: no skin bitmap, everything drawn by code from a few theme colours (uikit/theme.h), integer arithmetic only (the apps build with -mgeneral-regs-only).

```
* A shade from a colour: a grey profile -- level 128 = the colour itself, above
toward white,
    below toward black -- so any hue lightens as well as darkens (uk_tone).
* A gradient: a colour a row, computed at the size being drawn (a button stretches
at will).
* Rounded corners without a formula at draw time: per radius, a table made once of
each of the
    corner's rows' first pixel and the opacities of its partly covered pixels (the
anti-
    aliasing); a row is then a straight span plus a few pixels blended over what the
canvas
    holds -- so a widget first fills its canvas with its parent's background.
* The push button: a raised face (a gradient, a light top edge, an outline;
pressed: a shade
    darker), the same as a drop-down's box -- no frame round it.
* Small glyphs (a check, arrows, a cross...) from their geometry, anti-aliased by
coverage.
```

the canvas

The alpha mode: the canvas's pixels carry a transparency in their top byte (0 opaque .. 255 see-through: a WIN_FLAG_ALPHA window -- clear it to 0xFF000000 first); what is drawn over a see-through pixel keeps its own colour, partly see-through where it is anti-aliased. Off by default (an opaque canvas: the blends are over what it holds).

```
void uk_paint_alpha (bool on);
```

One pixel of c at opacity a (0..255) over what the canvas holds (the alpha mode heeded).

```
void uk_blend_px (Canvas &cv, int x, int y, unsigned c, int a);
```

colours

```
unsigned uk_tone (unsigned c, int level);           // the grey profile: 128 = c, 255 =
white, 0 = black
unsigned uk_mix (unsigned a, unsigned b, int t);    // t = 0 (a) .. 256 (b)
static inline unsigned uk_over (unsigned dst, unsigned c, int alpha) // c at
opacity 0..255 over dst
{ return uk_mix (dst, c, alpha + (alpha >> 7)); }
int      uk_bright (unsigned c);                   // 0..255 (0.30 R + 0.59 G + 0.11 B)
unsigned uk_ink_on (unsigned c);                   // dark or white text on c
```

shapes

The rounded corner of radius r (1..16): for each of its rows (the top one first), the x offset of its first pixel, the number of partly covered pixels from there and their opacities (0..255).

```
struct UkCorner { int r; unsigned char off[16], n[16], a[16][16]; };
const UkCorner &uk_corner (int r); // the table for radius r (clamped to 1..16),
computed once and kept
```

A rounded box [x, x + w) x [y, y + h), corners of radius r (0 = square; which ones: UK_TL | UK_TR | UK_BL | UK_BR, all by default), filled with a vertical gradient from top (its first row) to bottom (its last), at opacity alpha (255 = opaque); the corners' edges blended over what the canvas holds.

```
enum { UK_TL = 1, UK_TR = 2, UK_BL = 4, UK_BR = 8, UK_ALL = 15 };
void uk_rbox (Canvas &cv, int x, int y, int w, int h, int r, unsigned top, unsigned
bottom,
             int alpha = 255, int corners = UK_ALL); // fill a rounded box with a
vertical gradient, top to bottom
```

Its 1-px outline in colour c at opacity alpha.

```
void uk_rline (Canvas &cv, int x, int y, int w, int h, int r, unsigned c, int alpha =
255,
              int corners = UK_ALL);
```

the look's pieces

```
enum { UK_NORMAL = 0, UK_HOT = 1, UK_PRESSED = 2, UK_DISABLED = 3, UK_FOCUS = 0x10 };
```

The push button in the box, from its face colour: a raised face as a drop-down's (uk_raised; it had a frame and a well round it: the name); returns the label's box in *bx..*bh (already 1 px lower right when pressed) if the pointers are given.

```
void uk_framed (Canvas &cv, int x, int y, int w, int h, unsigned face, int state,
               int *bx = 0, int *by = 0, int *bw = 0, int *bh = 0);
```

A raised face (a header, a tab, a handle, a scroll bar's thumb): a gradient, a light top edge, an outline.

```
void uk_raised (Canvas &cv, int x, int y, int w, int h, int r, unsigned face, int
state = UK_NORMAL);
```

A sunken field (a text box, a list, a display): its background, a shadow along the top, an outline -- the accent's when focused.

```
void uk_sunken (Canvas &cv, int x, int y, int w, int h, int r, unsigned bg, bool
focus = false);
```

An etched line (a groove: dark over light) across a face.

```
void uk_etch_h (Canvas &cv, int x, int y, int w, unsigned face);
void uk_etch_v (Canvas &cv, int x, int y, int h, unsigned face);
```

An etched rounded frame (a group box's), the same groove all round.

```
void uk_etch_box (Canvas &cv, int x, int y, int w, int h, int r, unsigned face);
```

The controls' marks, s x s at (x, y) (state: UK_NORMAL / HOT / PRESSED / DISABLED | FOCUS): a check box (a sunken rounded field; checked: the accent, a white check), a radio button (the same, round; checked: a white dot on the accent), a switch (a pill: the accent when on, a knob that slides) w x h.

```
void uk_check_mark (Canvas &cv, int x, int y, int s, bool checked, int state);
void uk_radio_mark (Canvas &cv, int x, int y, int s, bool checked, int state);
void uk_switch_mark (Canvas &cv, int x, int y, int w, int h, bool on, int state);
```

A scroll bar in the box: a groove (the track, a shade of bg), the thumb a raised pill at [pos, pos + len) along it (state: UK_HOT while pointed / dragged).

```
void uk_scroll_bar (Canvas &cv, int x, int y, int w, int h, bool vertical, int pos,
int len,
    unsigned bg, int state = UK_NORMAL);
```

A slider's groove (the accent up to fill px) and its knob at kx (a raised rounded thumb), across w x h.

```
void uk_slider_mark (Canvas &cv, int x, int y, int w, int h, int fill, int kx, int
kw, int state);
```

A bar that fills (a progress bar): a sunken track, fill px of the accent's gradient.

```
void uk_progress_bar (Canvas &cv, int x, int y, int w, int h, int fill);
```

A floating panel (a drop-down list, a menu, a drawer, a tooltip): a light face, rounded, an outline; its corners' outside set to the magenta key (the widget is blitted transparent: what lies below shows there), their inside blended toward the outline.

```
void uk_popup (Canvas &cv, int x, int y, int w, int h, int r, unsigned face);
```

Set the pixels of the box's rounded corners that are mostly outside to the magenta key.

```
void uk_corner_key (Canvas &cv, int x, int y, int w, int h, int r);
```

A highlighted row (a selection, the item under the pointer): the accent, rounded; dimmer when its list does not have the focus. The text on it: uk_hilite_ink.

```
void uk_hilite (Canvas &cv, int x, int y, int w, int h, int r, bool strong = true);
unsigned uk_hilite_ink (bool strong = true);
```

A small window-like title strip (a dialog's): the active frame's colour, a gradient (Milk's down to C_FACE: the box under it, the dialog's face), its text bold and centred.

```
void uk_title_strip (Canvas &cv, int x, int y, int w, int h, const char *s, int r =
0);
```

A glossy bead d x d at (x, y) in colour c (the Milk theme's title buttons, as OS X's): a sphere of gel -- darker at the top, lit at the bottom, a white gloss over its upper half, a dark rim; anti-aliased over what the canvas holds.

```
void uk_bead (Canvas &cv, int x, int y, int d, unsigned c);
```

glyphs

```
enum { WKG_CHECK, WKG_UP, WKG_DOWN, WKG_LEFT, WKG_RIGHT, WKG_CLOSE, WKG_MIN, WKG_MAX,
WKG_MENU,
      WKG_DOT, WKG_PLUS, WKG_MINUS, WKG_RESTORE, WKG_CHEV_UP, WKG_CHEV_DOWN,
WKG_CHEV_LEFT,
      WKG_CHEV_RIGHT, WKG_RING, WKG_LOCK, WKG_GEAR, WKG_POWER,
      WKG_RELOAD, WKG_HOME,      // (a circular arrow, a house: a browser's toolbar)
      WKG_HISTORY };             // (a clock: a browser's history)
```

A glyph centred on (cx, cy), about size px across, in colour c.

```
void uk_glyph (Canvas &cv, int kind, int cx, int cy, int size, unsigned c);
```

text

The theme's text in a box: centred, or left-aligned at x (vertically centred).

```
void uk_text_c (Canvas &cv, int x, int y, int w, int h, const char *s, unsigned c,
int style = 0);
void uk_text_l (Canvas &cv, int x, int y, int h, const char *s, unsigned c, int style
= 0);
```

The text colour for a background: C_TEXT on the window's face and on what is as light (or as dark), else black or white (uk_ink_on): a label on a program's own colour stays readable.

```
unsigned uk_ink_for (unsigned bg);
int uk_text_w (const char *s, int style = 0);
```

uikit/panel.h

uikit/panel.h -- a plain container with a solid background; holds child widgets.

```
class Panel : public Widget
{
public:
    unsigned bg;
    Panel (int l, int t, int w, int h, unsigned bg_ = C_BG);
    unsigned bgColor () override { return bg; }
    void onDraw () override;
};
```

uikit/progress.h

uikit/progress.h -- non-interactive bar, fills proportionally.

```
class Progress : public Widget
{
public:
    int value, vmin, vmax;
    Progress (int l, int t, int w, int h, int lo, int hi, int val);
    void setValue (int v);
    void onDraw () override;
};
```

uikit/radio.h

uikit/radio.h -- RadioButton: a round button + label; the buttons of the same group under the same parent are exclusive (selecting one clears the others). cb fires on selection. GroupBox (groupbox.h) is the usual container for a set of them.

```

class RadioButton : public Widget
{
public:
    char    text[64]; int group; bool checked; Action cb; unsigned bg;
    RadioButton (int l, int t, int w, int h, const char *s, int group_, bool chk,
Action cb_, unsigned bg_ = C_BG);
    void select ();                // check me, uncheck my group's siblings, fire cb
    void onDraw () override;
    bool onMouse (int mx, int my, int bl, int br, int bm, int wheel) override;
    bool onKey (long k) override;  // Space / Enter select
    RadioButton *asRadio () override { return this; }
};

```

The checked button of `group` among parent's children (0 = none).

```
RadioButton *uk_radio_checked (Widget *parent, int group);
```

uikit/richtextbox.h

uikit/richtextbox.h -- multi-line *styled* text editor (word processor body). Unlike Textarea (one colour, one size), every character carries its own style: bold / italic / underline / strikethrough / highlight, a 16-colour foreground + highlight palette, and an integer size multiplier (x1..x8, smoothed). Paragraphs carry a heading "level" (Normal / Titre 1-3) applied as a preset across the paragraph.

All styling is synthesised user-side from Onyx's single monospace bitmap font: the glyph mask is fetched via `kapi_draw_text_buf`, then emboldened / sheared / bilinearly upsampled and blended -- no kernel/ABI change, no FreeType. Default page is white, text black. The widget owns its text buffer + a parallel per-char attribute buffer.

character style flags (bits 0..7 of the packed attribute)

```

enum {
    RT_BOLD    = 0x01,
    RT_ITALIC  = 0x02,
    RT_UNDER   = 0x04,
    RT_STRIKE  = 0x08,
    RT_HILITE  = 0x10,    // draw the highlight (background) band
};

```

16-colour palette indices (see RT_PAL[] in the .cpp)

```

enum {
    RT_BLACK = 0, RT_WHITE, RT_GRAY, RT_SILVER,
    RT_RED,   RT_GREEN, RT_BLUE, RT_YELLOW,
    RT_CYAN,  RT_MAGENTA, RT_ORANGE, RT_PURPLE,
    RT_DKRED, RT_DKGREEN, RT_DKBLUE, RT_TEAL,
};

```

paragraph heading levels (presets applied by setLevel)

```
enum { RT_NORMAL = 0, RT_TITLE1, RT_TITLE2, RT_TITLE3 };
```

Unpacked character style (public API + the "current typing style"). Packs into one u32 stored per character, parallel to the text buffer.

```

struct RtStyle
{
    unsigned char flags;    // RT_BOLD | RT_ITALIC | ...
    unsigned char fg;      // foreground palette index
    unsigned char bg;      // highlight palette index (used when RT_HILITE set)
};

```

```

    unsigned char size; // 1..8 (font cell multiplier)
    RtStyle () : flags (0), fg (RT_BLACK), bg (RT_YELLOW), size (1) {}
};

unsigned rt_pack  (const RtStyle &s);    // RtStyle -> packed u32
RtStyle  rt_unpack (unsigned a);        // packed u32 -> RtStyle
unsigned rt_color (int paletteIndex);   // palette index -> 0x00RRGGBB

class RichTextBox : public Widget
{
public:
    char      *buf;    // text ('\n' between paragraphs), NUL-terminated
    unsigned  *attr;    // packed RtStyle per character (parallel to buf)
    int       cap, len, caret, sel; // sel = selection anchor (-1 = none)
    int       top;     // first visible VISUAL (wrapped) row
    int       goalX;    // desired caret x for vertical moves (pixels, row-relative)
    bool      readonly;
    RtStyle   cur;     // style stamped onto newly typed characters

    RichTextBox (int l, int t, int w, int h, int capacity);
    ~RichTextBox () override;

    void      setContent (const char *s); // load plain text in the default style
    const char *content () const { return buf; }
    int      length () const { return len; }

    // ---- styling: applies to the selection, else to `cur` (next typed text) ----
    void toggleFlag (unsigned flag);    // bold/italic/underline/strike/hilite
    void setFg      (int paletteIndex);
    void setHilite  (int paletteIndex); // set RT_HILITE + highlight colour
    void clearHilite ();
    void setSize   (int mult);         // 1..8
    void setLevel   (int level);       // RT_NORMAL / RT_TITLE1.. on caret
    paragraph(s)
    void selectAll  ();

    // ---- clipboard support (plain text) ----
    bool hasSelection () const { return hasSel (); }
    int  selectedText (char *out, int cap) const; // copy the selection, NUL-
    terminated
    void cutSelection ();                // delete the selection
    void insertText  (const char *s);    // replace the selection / insert at the
    caret
    RtStyle caretStyle () const;         // style at the caret (for toolbar state)

    // ---- scrolling (a host Scrollbar wires value<->topRow, vmax<->rowCount-1) ----
    int  rowCount ();                   // total visual rows
    int  topRow () const { return top; }
    void setTopRow (int r);

    void onDraw () override;
    bool onMouse (int mx, int my, int bl, int br, int bm, int wheel) override;
    bool onKey (long k) override;

private:
    // text buffer
    bool grow (int need);
    void insertChar (char ch);
    void deleteRange (int a, int b);    // remove [a,b)

```

```

bool hasSel () const { return sel >= 0 && sel != caret; }
void selRange (int &a, int &b) const;
void deleteSelection ();

// paragraph / glyph metrics
int paraStart (int i) const { while (i > 0 && buf[i - 1] != '\n') i--; return i; }
}
int paraEnd (int i) const { while (i < len && buf[i] != '\n') i++; return i; }
int glyphAdvance (int i) const;

// word-wrap layout cache (rebuilt lazily on edit / resize)
int *rowStart, *rowH, *rowY; // rowY has rowN+1 entries (prefix tops + total)
int rowN, rowCap, lastWrapW;
bool layoutDirty;
bool barDrag; // dragging the integrated scrollbar thumb
void ensureLayout ();
void relayout (int wrapW);
void rowAt (int s, int wrapW, int &end, int &h, int &nextStart, bool &hard)
const;
bool growRows (int need);
int rowOfChar (int c);
int xInRow (int pos);
void ensureVisible ();
void moveVert (int dir);
int hitTest (int mx, int my);

// styled glyph rasteriser
void drawGlyph (int px, int py, char ch, const RtStyle &st);
};

```

uikit/scrollbar.h

uikit/scrollbar.h -- draggable thumb (vertical or horizontal), value in [0,vmax].

```

class Scrollbar : public Widget
{
public:
    bool vertical; int value, vmax; Action cb;
    Scrollbar (int l, int t, int w, int h, bool vert, int maxv, int val, Action cb_);
    void setFromXY (int px, int py);
    void scrollBy (int units); // clamp value+=units, fire cb (wheel / keyboard)
    void onDraw () override;
    bool onMouse (int mx, int my, int bl, int br, int bm, int wheel) override;
};

```

uikit/segmented.h

uikit/segmented.h -- SegmentedControl: a row of mutually exclusive labelled segments drawn as one pill (a radio group, a small tab strip): the chosen one filled with the accent. Click a segment, or Left / Right when focused; onChange fires when selected changes. The labels are copied.

```

static const char *const MODES[] = { "Articulation", "Melodic cell", "Voicing" };
SegmentedControl *sc = new SegmentedControl (x, y, 300, 26, MODES, 3, 0, onMode);

class SegmentedControl : public Widget
{
public:
    static const int MAXSEG = 12;
    int selected; // the chosen segment (-1: none)
    bool equalWidths; // every segment as wide (default); false: each
its text's width,

```

```

        // the room left shared out
        Action    onChange;

        SegmentedControl (int l, int t, int w, int h, const char *const *labels, int n,
int sel = 0, Action cb = 0);
        void setLabels (const char *const *labels, int n);
        void select (int i, bool fire = false); // repainted; onChange if fire and it
changed
        int count () const { return m_n; }
        const char *label (int i) const { return i >= 0 && i < m_n ? m_label[i] : ""; }
        void setEnabled (int i, bool on); // a segment greyed out (not chosen by a
click)
        int segmentAt (int mx) const; // the segment under x (-1: none)

        void onDraw () override;
        bool onMouse (int mx, int my, int bl, int br, int bm, int wheel) override;
        bool onKey (long k) override;
private:
        char    m_label[MAXSEG][32];
        bool    m_off[MAXSEG];
        int m_n, m_hot, m_x[MAXSEG + 1]; // the segments' edges (m_x[i] .. m_x[i + 1])
        void place ();
};

```

uikit/skin.h

uikit/skin.h -- 9-slice bitmap skin (port of uikit's Skin / the kernel CSkin) plus user-side window decoration (drawn by code: uikit/paint.h). A skin BMP holds `count` states stacked vertically (button.bmp: normal/hover/pressed); margins mark the fixed corners/edges, the middle tiles. Magenta (UK_TRANSPARENT_KEY) is the transparency key. Skins draw into RAW 0x00RRGGBB buffers -- a Canvas's `px`, or a window-chrome buffer from `uk_win_chrome`.

Multiply a 0x00RRGGBB pixel by a 0x00RRGGBB tint (per channel /255). 0xFFFFFFFF = no-op.

```
unsigned uk_tint (unsigned c, unsigned t);
```

```

class Skin
{
public:
    unsigned *pix; int imgW, imgH, count, sw, sh, ml, mr, mt, mb;
    Skin ();
    ~Skin ();
    bool valid () const { return pix != 0 && sh > 0; }
    bool load (const char *path, int cnt, int l, int r, int t, int b);
    // Blit a magenta-keyed sub-rect of the skin into a raw target (fb, W x H).
    void blit (unsigned *fb, int W, int H, int sx, int sy, int bw, int bh,
        int dx, int dy, unsigned tint = 0xFFFFFFFF);
    // 9-slice draw of `state` into the (x,y,w,h) box: fixed corners, tiled
edges/centre.
    void drawOn (unsigned *fb, int W, int H, int state, int x, int y, int w, int h,
        unsigned tint = 0xFFFFFFFF);
};

```

Draw the window's frame (the modernised CDE: kapi v64 -- a gradient from the theme's frame colour, rounded corners, the theme's outline, the title buttons: the window menu, minimise, maximise, close; the title in bold) into both chrome copies of this window. No-op for a borderless window / no window. Drawn once per size, title, theme and state (guarded); Root calls it after creating its window; call it again after `uk_win_resize` (the frame then follows the new size).

```
void uk_decorate_window ();
```

The frame's state, drawn by `uk_decorate_window` (Root keeps it): the window can be maximised (its maximise button active), it is maximised (the button shows "restore"), the app answers the window menu (GUI_EVENT_WINCTL, as a Root does: else its button is greyed -- an app drawing its own window without a Root).

```
enum { UK_WIN_RESIZABLE = 1, UK_WIN_MAXIMISED = 2, UK_WIN_MENU = 4, UK_WIN_FIXED = 8
}; // (FIXED: no buttons)
void uk_window_state (int flags); // set the frame's state (UK_WIN_* flags) for the
next uk_decorate_window
int uk_window_flags (); // the UK_WIN_* flags last set (0 at start)
```

A frame as `uk_decorate_window` draws it, into any W x H buffer (T: its title bar's height), in the colour `frame` -- a preview (the Theme app). Its corners' outside: see-through (top byte).

```
void uk_draw_frame (unsigned *fb, int W, int H, int T, const char *title, unsigned
frame, bool active);
```

uikit/slider.h

uikit/slider.h -- horizontal value in [vmin,vmax]; drag the thumb.

```
class Slider : public Widget
{
public:
    int value, vmin, vmax; Action cb; unsigned bg;
    Slider (int l, int t, int w, int h, int lo, int hi, int val, Action cb_, unsigned
bg_ = C_BG);
    void setFromX (int px);
    void onDraw () override;
    bool onMouse (int mx, int my, int bl, int br, int bm, int wheel) override;
};
```

uikit/splitter.h

uikit/splitter.h -- a two-pane container with a draggable divider ("grip"). Add the two content panes with `setPanes()`; the splitter sizes them and paints/handles a GRIP-px divider between, re-laying out automatically when it is resized (so a parent collapse/resize cascades straight through). Dragging the grip moves the split. Two orientations:

```
HSplitter -- panes side by side (vertical grip, drag left/right)
VSplitter -- panes stacked (horizontal grip, drag up/down)
```

```
class SplitGrip; // internal draggable divider (defined in splitter.cpp)
```

```
class Splitter : public Widget
{
public:
    bool vertical; // true = stacked (VSplitter); false = side by side
(HSplitter)
    int split; // size of the FIRST pane along the split axis (px)
    int grip; // divider thickness (px)
    int minA, minB; // minimum pane sizes along the split axis
    unsigned gripCol, gripHi; // divider colour (idle / hover)
    unsigned bg; // backdrop (normally hidden: panes + grip tile us)
    unsigned bgColor () override { return bg; }

    Splitter (int l, int t, int w, int h, bool vertical_, int split_ = -1,
unsigned bg_ = C_BG);
    void setPanes (Widget *a, Widget *b); // adopt the two content panes (call
once)
```

```

    void setSplit (int s);           // move the divider (clamped to mins), relayout
    void layout () override;
    void onDraw () override;

private:
    Widget      *paneA, *paneB;
    SplitGrip   *gripW;
};

```

```

class HSplitter : public Splitter
{ public: HSplitter (int l, int t, int w, int h, int split = -1, unsigned bg = C_BG)
    : Splitter (l, t, w, h, false, split, bg) {} };

```

```

class VSplitter : public Splitter
{ public: VSplitter (int l, int t, int w, int h, int split = -1, unsigned bg = C_BG)
    : Splitter (l, t, w, h, true, split, bg) {} };

```

UIKit/sysclip.h

sysclip.h -- UIKit's own use of the system clipboard (its text fields' copy and paste): SystemKit, opened when a field first copies or pastes. (A program calls SystemKit's clip_* itself: systemkit/clipboard.h.)

```

bool uk_clip_ready ();           // SystemKit is there (false: no clipboard -- the
field does nothing)
}
#endif

```

UIKit/tabhost.h

UIKit/tabhost.h -- a section that hosts several "tasks" (apps) and shows ONE at a time, chosen from a popup menu in its header (the activity-shell tab strip, popup variant). addTab() adopts a content widget; the header Dropdown lists the tab titles; selecting one shows that content (the others are hidden, not destroyed). The content fills the area below the header and is resized with the host, so the hosted app stays passive about geometry.

```

class TabHost : public Panel
{
public:
    static const int MAXTABS = 8;
    int headerH;                 // height of the header strip (holds the popup)

    TabHost (int l, int t, int w, int h, int headerH_ = 26, unsigned bg = C_BG);
    int addTab (const char *title, Widget *content); // -> tab index, or -1 if
full
    void select (int i);          // show tab i (hide the previous)
    int count () const { return ntabs; }
    int current () const { return active; }
    void layout () override;
    void onDraw () override;

private:
    Dropdown *picker;             // the task popup menu
    Widget *content[MAXTABS];
    char titles[MAXTABS][32];
    const char *titlePtr[MAXTABS]; // stable pointers into titles[] for the
Dropdown
    int ntabs, active;
    static void onPick (Widget &w); // Dropdown cb -> select the picked tab
};

```

uikit/tabstrip.h

uikit/tabstrip.h -- TabStrip: a row of tabs over a view (a terminal's shells, a reader's documents): each a title, a close cross, a mark (a dot: something runs, something new), and a "+" after them. A click picks a tab; the wheel, or selectNext (Ctrl+Tab, Ctrl+PgUp / PgDn: the app's keys), steps through them. The cross or a middle click ASKS for a tab's close: onClose fires with `closing` set, and the app decides (a question first, its own state freed) and calls `remove ()`. The strip only shows the tabs: the app shows the chosen one's content. Each tab carries a pointer for the app (`data ()`), moved with it when a tab before it goes. The titles are copied; a long one is cut with an ellipsis.

```
TabStrip *ts = new TabStrip (0, 0, w, 30, onTab);
ts->onClose = onTabClose; ts->onNew = onNewTab;
int i = ts->add ("SD:/", myTab); ts->select (i);
```

MIT licence (Onyx).

```
class TabStrip : public Widget
{
public:
    static const int MAXTABS = 32;
    int selected;           // the chosen tab (-1: none)
    int closing;           // the tab whose close was asked (onClose's)
    bool closable;         // a close cross on each tab (true)
    bool newButton;        // the "+" after the tabs (true)
    unsigned activeFace;   // the chosen tab's face: the content's
background below it,
    Action onChange;       // which it opens onto (0: the window's, C_BG)
selectNext)
    Action onClose;       // a close asked: the cross, a middle click (closing
= the tab)
    Action onNew;         // the "+"

    TabStrip (int l, int t, int w, int h, Action change = 0);
    int add (const char *title, void *data = 0); // at the end -> its index (-1:
full)
    void remove (int i); // the tabs after it move down; selected follows
its tab
// (the removed one's: the tab now there, else the one
before)
    int count () const { return m_n; }
    void setTitle (int i, const char *title);
    const char *title (int i) const { return i >= 0 && i < m_n ? m_title[i] : ""; }
    void *data (int i) const { return i >= 0 && i < m_n ? m_data[i] : 0; }
    void setData (int i, void *p) { if (i >= 0 && i < m_n) m_data[i] = p; }
    void setMark (int i, bool on); // the dot before the title
    void select (int i, bool fire = false); // repainted; onChange if fire and it
changed
    void selectNext (int d, bool fire = true); // d = +1 / -1, round
    int tabAt (int mx) const; // the tab under x (-1: none)

    void onDraw () override;
    bool onMouse (int mx, int my, int bl, int br, int bm, int wheel) override;
private:
    char m_title[MAXTABS][48];
    void *m_data[MAXTABS];
    bool m_mark[MAXTABS];
    int m_n, m_hot, m_hotPart; // the tab under the pointer; 1 its cross, 2 the
"+"
}
```

```

    int m_x[MAXTABS + 1], m_plusX; // the tabs' edges (m_x[i] .. m_x[i + 1] - gap),
the "+"s
    bool m_mid; // the middle button held (one close a press)
    void place ();
    bool inCross (int i, int mx, int my) const;
};

```

uikit/textarea.h

uikit/textarea.h -- multi-line editable text (own '\n'-separated buffer), caret-driven vertical+horizontal scroll, click to position. Clipped to its own canvas. Selection: Shift + arrows / Home / End / Page Up / Down (kapi_get_modifiers), a mouse drag, Shift+click, Ctrl+A. Typing, Enter, Backspace and Delete replace / remove it; Ctrl+C / Ctrl+X / Ctrl+V copy / cut / paste through the system clipboard (unless the app's menu takes those shortcuts first -- then it can call copy () / cut () / paste ()). With a text face installed (uikit/text.h) the text is UTF-8 and the caret, the clicks and the selection follow the glyphs' widths (the horizontal scroll is then leftPx, in pixels).

```

class Textarea : public Widget
{
public:
    char *buf; int cap, len, caret, top, left, rows, cols; bool readonly, barDrag;
    int leftPx; // with a text face: the horizontal scroll, px
    int anchor; // selection = [anchor, caret) either way; -1 = none
    // Own colours (0x00RRGGBB) instead of the theme's: e.g. QBasic's grey on blue.
    bool ownColors; unsigned colBg, colText, colCaret, colSel;
    void setColors (unsigned bg, unsigned text, unsigned caretC, unsigned sel)
{ ownColors = true; colBg = bg; colText = text; colCaret = caretC; colSel = sel;
invalidate (true); }
    Textarea (int l, int t, int w, int h, int capacity);
    ~Textarea () override;
    const char *content () const { return buf; }
    void setContent (const char *s);
    void insertText (const char *s); // insert at the caret (clipboard paste)
    void gotoLine (int line); // caret to the start of line (0-based),
scrolled into view
    int caretLine () const; // 0-based line / column of the caret
    int caretCol () const { return caret - lineStart (caret); }
    bool hasSelection () const { return anchor >= 0 && anchor != caret; }
    int selStart () const { return hasSelection () ? (anchor < caret ? anchor :
caret) : caret; }
    int selEnd () const { return hasSelection () ? (anchor < caret ? caret : anchor)
: caret; }
    int selectedText (char *out, int cap) const; // copies the selection; returns
its length
    void deleteSelection ();
    void selectAll ();
    void copy (); // selection -> clipboard
    void cut ();
    void paste (); // clipboard text -> caret (replaces the
selection)
    void onDraw () override;
    bool onMouse (int mx, int my, int bl, int br, int bm, int wheel) override;
    bool onKey (long k) override;
private:
    int lineStart (int i) const { while (i > 0 && buf[i - 1] != '\n') i--; return i; }
    int lineEnd (int i) const { while (buf[i] && buf[i] != '\n') i++; return i; }
    void insertAt (int ch);
    void moveCaret (long k); // one navigation key (no selection logic)
    void deleteAt (int i);
};

```

```

    void ensureVisible (int vr, int vc);
    // With a proportional face installed (uikit/text.h): UTF-8, the caret and the
    clicks by measure.
    int lineW (int ls, int i);           // px from the line's start ls to byte i
    int placeAt (int ls, int x);         // the byte of line ls nearest x px (text
    coordinates)
    int placeRow (int row, int mx);      // the byte at mx on the row shown `row`
    void drawFace ();
};

```

uikit/textbox.h

uikit/textbox.h -- single-line editable field; click to position caret, type to edit; Ctrl+V pastes the clipboard at the caret, Ctrl+C / Ctrl+X copy / cut the whole field (not a password). (With a text face installed -- uikit/text.h -- its text is UTF-8 and the caret follows the glyphs' widths.)

```

class Textbox : public Widget
{
public:
    enum { TEXT_CAP = 512 };           // the buffer; how much of it is used: maxLen
    char text[TEXT_CAP]; int caret; bool password; Action cb;
    int maxLen;                        // the most bytes typed or set (63; up to TEXT_CAP - 1: a
    chat line)
    int padR;                          // px kept free at the right (a Combobox's arrow)
    int vstart;                        // the first character shown (set by onDraw)
    Action changed;                    // called after each edit typed or pasted (0: none;
    cb is Enter's)
    Textbox (int l, int t, int w, int h, const char *s = "", Action cb_ = 0);
    void setText (const char *s);
    void onDraw () override;
    bool onMouse (int mx, int my, int bl, int br, int bm, int wheel) override;
    bool onKey (long k) override;
    bool isField () override { return true; } // a Tab stop
    void onTabFocus () override;           // Tab: the caret at the end
private:
    bool editKey (long k);
    // With a proportional face installed (uikit/text.h): the text is UTF-8,
    measured; vstart is then
    // the byte the field shows from.
    int shown (char *d) const;           // what is shown (a password's '*'s) -> its
    length
    int shownAt (int i) const;           // byte i of the text -> its place in what is
    shown
    int textAt (int d) const;           // ... and back
    void drawFace ();
    bool keyFace (long k);
    bool clipKey (long k);              // Ctrl+C / X / V -> true if taken
};

```

uikit/toggle.h

uikit/toggle.h -- ToggleSwitch: an on/off pill switch + label (WPF-style ToggleButton / UWP ToggleSwitch). Click (or Space) flips it; cb fires; read on.

```

class ToggleSwitch : public Widget
{
public:
    char text[64]; bool on; Action cb; unsigned bg;
    ToggleSwitch (int l, int t, int w, int h, const char *s, bool on_, Action cb_,
    unsigned bg_ = C_BG);
    void setOn (bool v) { if (v != on) { on = v; invalidate (true); } }
};

```

```

    void onDraw () override;
    bool onMouse (int mx, int my, int bl, int br, int bm, int wheel) override;
    bool onKey (long k) override;
};

```

uikit/toolbar.h

uikit/toolbar.h -- ToolBar + ToolButton: a strip of small buttons (generalised from Letters'): an icon (one of the WKT_* set below, drawn from its geometry at any size, or the app's own drawer), a text label beside it or alone, a toggle state (lit: the accent's tint, or filled -- a play button), a split part (an arrow opening a palette or a menu), a tooltip; separators and gaps.

```

#include "uikit/toolbar.h"          (NOT in uikit/uikit.h: Letters, the Spreadsheet
and Cardfile have                  their own ToolBar / ToolButton and `using namespace
uikit`)
    ToolBar *tb = new ToolBar (0, 0, W, 38);
    tb->add ((new ToolButton (30, 28, "Save (Ctrl+S)", onSave))->setGlyph (WKT_SAVE));
    tb->sep ();
    ToolButton *play = (new ToolButton (34, 28, "Play (Space)", onPlay))->setGlyph
(WKT_PLAY)->setToggle (true);
    play->filled = true;  tb->add (play);
    tb->add ((new ToolButton (0, 28, "Loop"))->setGlyph (WKT_LOOP)->setText ("Loop")-
>fitWidth ());

```

onClick gets the button (a toggle's on already flipped). A button is flat until pointed (raised: always a face, as a transport's); it does not take the keyboard focus.

The toolbar icons, drawn in a size x size box at (x, y) in ink (anti-aliased, uikit/vpaint.h).

```

enum { WKT_NONE = -1, WKT_NEW = 0, WKT_OPEN, WKT_SAVE, WKT_UNDO, WKT_REDO, WKT_CUT,
WKT_COPY, WKT_PASTE,
    WKT_PLAY, WKT_PAUSE, WKT_STOP, WKT_RECORD, WKT_TO_START, WKT_TO_END,
WKT_REWIND, WKT_FORWARD,
    WKT_LOOP, WKT_METRONOME, WKT_PLUS, WKT_MINUS, WKT_SEARCH, WKT_MIXER,
WKT_SPARK, WKT_GEAR,
    WKT_TRASH, WKT_PIN,          // (2026-10-06: a waste bin -- delete to the Trash
-- , a push pin -- on the desktop)
    WKT_COUNT };
void uk_tool_glyph (Canvas &cv, int kind, int x, int y, int size, unsigned ink);
// draw the icon `kind` (WKT_*); size: 6 px at least

```

An app's icon: drawn in the size x size box at (x, y), ink the text's colour (greyed when off).

```

typedef void (*ToolIconFn) (Canvas &cv, int id, int x, int y, int size, unsigned ink,
bool off);

```

```

class ToolButton : public Widget
{
public:
    int    glyph;          // a WKT_* icon (WKT_NONE: none)
    ToolIconFn iconFn; int iconId;      // ... or the app's drawer
    int    iconSize;       // the icon's box, px (18)
    char    text[32];       // a label (right of the icon; alone: a text
button)
    bool    toggle, on;     // a toggle: a click flips `on`; on: lit
    bool    filled;         // on: filled with onColor (a play button) rather
than tinted
    bool    raised;         // a face even when not pointed (a transport's
buttons)
    unsigned onColor;       // UK_AUTO: the accent

```

```

    unsigned    iconColor;           // the icon's own colour (UK_AUTO: the text's): a
record's red
    Action      onClick;
    void        (*arrow) (ToggleButton &); // a split button's arrow part (0: none)

ToggleButton (int w, int h, const char *tip = 0, Action cb = 0);
ToggleButton *setGlyph (int kind);
ToggleButton *setIcon (ToolIconFn fn, int id);
ToggleButton *setText (const char *s);
ToggleButton *setToggle (bool isToggle, bool isOn = false);
ToggleButton *setSplit (void (*arrowCb) (ToggleButton &)); // (adds the arrow part's
width)
ToggleButton *fitWidth ();           // as wide as its icon and label need
void setOn (bool v);
void setDisabled (bool d);

void onDraw () override;
bool onMouse (int mx, int my, int bl, int br, int bm, int wheel) override;
static const int SPLIT_W = 13;
private:
    int m_part, m_down;              // the part under the pointer, the one pressed (-1
none; 1 the arrow)
    unsigned face ();
};

class ToolBar : public Widget
{
public:
    unsigned bg;                     // its face (UK_AUTO: the window's, C_BG)
    bool line;                       // an etched line along its bottom
    ToolBar (int l, int t, int w, int h = 34);
    void add (Widget *w, int gap = 1); // at the next place (after a gap), centred
vertically
    void addRight (Widget *w, int gap = 4); // from the right end leftward (they
follow a resize)
    void sep ();                     // an etched separator
    void space (int px) { m_x += px; } // a gap
    int next () const { return m_x; } // where the next one goes
    unsigned bgColor () override { return bg == UK_AUTO ? C_BG : bg; }
    void onDraw () override;
private:
    int m_x, m_rx, m_nsep, m_sepX[24];
};

```

uikit/treeview.h

uikit/treeview.h -- TreeView: a hierarchy of labelled nodes with expand / collapse (WPF TreeView). add (parent, label) returns the node id (parent -1 = a root node). Click the [+] / [-] box (or double-click, or Right / Left) to expand / collapse; click selects (onSelect), double-click on a leaf or Enter activates (onActivate); Up / Down move; the wheel and the scrollbar scroll. Read sel (node id, -1 = none).

```

class TreeView : public Widget
{
public:
    int sel, top; Action onSelect, onActivate;
    TreeView (int l, int t, int w, int h, Action onSelect_ = 0, Action onActivate_ =
0);
    ~TreeView ();
    int add (int parent, const char *label);

```

```

    void clear ();
    void expand (int id, bool open);
    bool expanded (int id) const { return id >= 0 && id < m_n && m_nodes[id].open; }
    bool hasChildren (int id) const;
    const char *label (int id) const { return (id >= 0 && id < m_n) ?
m_nodes[id].label : ""; }
    int parentOf (int id) const { return (id >= 0 && id < m_n) ? m_nodes[id].parent
: -1; }
    int userData (int id) const { return (id >= 0 && id < m_n) ? m_nodes[id].data :
0; }
    void setUserData (int id, int d) { if (id >= 0 && id < m_n) m_nodes[id].data = d;
}

    void onDraw () override;
    bool onMouse (int mx, int my, int bl, int br, int bm, int wheel) override;
    bool onKey (long k) override;
private:
    struct Node { char label[48]; int parent, depth, data; bool open; };
    Node *m_nodes; int m_n, m_cap;
    int *m_vis; int m_nvis; // visible node ids, in display order
    unsigned m_lastClick; int m_lastRow; bool m_thumb;
    void rebuild ();
    void walk (int parent);
    int rows () const;
    int rowOf (int id) const;
    void scrollTo (int t);
    void pick (int id, bool fire);
};

```

uikit/vpaint.h

uikit/vpaint.h -- shapes from their geometry, anti-aliased: a toolbar's icons (a page, a folder, a floppy, arrows, scissors...), markers, anything drawn at any size without a bitmap. A VPath gathers outlines -- polygons, rectangles (rounded), discs, strokes (a line, a polyline, an arc: round ends and joins) -- then fill () paints their union in one colour over the canvas (the non-zero rule: the shapes added are all turned the same way, a hole () the other way). Coordinates are in 1/16 px (V (x) = x * 16), integers: the apps build without the FPU.

```

VPath p;
p.rrect (V (2), V (1), V (12), V (14), V (2)); p.fill (cv, 0xFFFFFFFF);
p.clear (); p.line (V (4), V (5), V (11), V (5), V (1)); p.fill (cv, ink);

```

```
int V (int px);
```

sin and cos of an angle in degrees, x 16384.

```

int uk_sin (int deg);
int uk_cos (int deg);

```

```

class VPath
{
public:
    VPath ();
    ~VPath ();
    void clear ();
    // Outlines (closed): a polygon of n points (xy: x0, y0, x1, y1...), a box, a
rounded box,
    // a disc, a hole (a disc cut out of what is under it), an ellipse.
    void poly (const int *xy, int n);
    void rect (int x, int y, int w, int h);
    void rrect (int x, int y, int w, int h, int r);

```

```

void circle (int cx, int cy, int r);
void hole (int cx, int cy, int r);
void ellipse (int cx, int cy, int rx, int ry);
// Strokes w wide: a line, a polyline (closed: a loop), an arc from a0 to a1
degrees
// (counter-clockwise from 3 o'clock, y down: 90 = 12 o'clock... as on paper).
void line (int x0, int y0, int x1, int y1, int w);
void polyline (const int *xy, int n, int w, bool closed = false);
void arc (int cx, int cy, int r, int a0, int a1, int w);
// An arrow head at (x, y) pointing toward angle deg, `len` long, `half` wide
each side.
void arrowHead (int x, int y, int deg, int len, int half);
// Paint it: c at opacity alpha, shifted by (dx, dy) px.
void fill (Canvas &cv, unsigned c, int alpha = 255, int dx = 0, int dy = 0);
private:
int *m_x, *m_y, m_n, m_cap; // the points
int *m_cs, m_nc, m_ccap; // each outline's first point
void add (int x, int y);
void begin ();
void end (bool asHole = false); // close the outline (turned the right way)
};

```

uikit/vumeter.h

uikit/vumeter.h -- VuMeter: a level meter (stereo or mono, vertical or horizontal) as a studio's: segments on a dark well, green, then amber, then red toward the top of the scale; a peak held a moment then falling; a clip light (a level over 0 dBFS; a click clears it). Levels are linear (1.0 = 0 dBFS, may go over), mapped to dB inside; the bars fall back at fallDb dB a second.

```

VuMeter *m = new VuMeter (x, y, 14, 120); // (vertical, stereo)
... at the UI's rate (Root::onTick): m->setQ16 (peakL, peakR); // (65536 = 1.0)
... or, in an FP-enabled translation unit: m->set (0.5f, 0.25f);

```

Call it regularly, silence included (set (0, 0)): the falls and the peaks' hold follow the clock (kapi_get_ticks) at each call. Cheap: it repaints only when a lit segment or a peak moves.

A linear level (65536 = 1.0 = 0 dBFS) → 1/100 dB (0 → -20000).

```
int uk_q16_to_cdb (unsigned q16);
```

```

class VuMeter : public Widget
{
public:
    bool vertical, stereo;
    int floorDb, topDb; // the scale, dB (-48 .. +6)
    int amberDb, redDb; // where the segments turn amber, then red (-12, -3)
    int segPx; // a segment's length with its 1-px gap (3; 0: a
continuous bar)
    int holdTicks; // the peak's hold (150 = 1.5 s at 100 Hz)
    int fallDb, peakFallDb; // the bar's, the peak's fall after its hold, dB
a second (26, 14)
    bool showPeak, showClip;
    bool clip[2]; // a level went over 0 dBFS (a click on the meter
clears them)

```

```

VuMeter (int l, int t, int w, int h, bool vertical = true, bool stereo = true);
void setQ16 (int l, int r); // linear levels, 65536 = 0 dBFS (mono: l)
void setCdb (int l, int r); // levels in 1/100 dB
void reset (); // all down, the peaks and clips cleared
int levelCdb (int ch) const { return m_lv1[ch & 1]; } // shown, 1/100 dB

```

```

    int peakCdb (int ch) const { return m_pk[ch & 1]; }
#ifdef defined (__ARM_FP) || !defined (__aarch64__) // (an FP-enabled unit: uikit
itself is integer-only)
    void set (float l, float r)
    {
        if (l < 0) l = -l;
        if (r < 0) r = -r;
        setQ16 (l > 16.f ? 1 << 20 : (int) (l * 65536.f), r > 16.f ? 1 << 20 : (int)
(r * 65536.f));
    }
#endif

    void onDraw () override;
    bool onMouse (int mx, int my, int bl, int br, int bm, int wheel) override;
private:
    int m_lvl[2], m_pk[2]; // shown level, peak (1/100 dB)
    unsigned m_pkT[2], m_last; // when each peak was set; the last call (ticks)
    int m_drawn[5]; // what the canvas shows (lit lengths, peak places,
clips)
    int len () const; // the scale's length, px
    int pos (int cdb) const; // a level's place along it, px
    unsigned colourAt (int cdb) const;
    void state (int *s) const;
};

```